B.Sc. (Hons) in Software Development

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

EchoChamber: A Multi-Account Platform for Analysing Algorithmic Personalisation and Political Content on X

By
Trinity Villaruel

April 26, 2026

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Abstract

Social media platforms increasingly rely on algorithmic content personalisation to determine the information presented to users. This has raised concerns regarding the potential amplification of politically polarised content and the formation of echo chambers. This project investigates whether gender-related account signals influence the political composition of content recommended by X. To address this, a platform named EchoChamber was developed to support longitudinal analysis of the X recommendation feed. The system consists of a Chrome extension for automated data capture, a FastAPI backend for processing, MongoDB for structured data storage, and a Streamlit interface for visualisation. The platform enables controlled experimentation by capturing and analysing content from multiple accounts over time. A multi-phase experimental methodology was adopted using two controlled accounts that remained identical except for gender-related variables. Across successive phases, changes such as gender assignment, username variation, interaction behaviour, and following politically aligned accounts were introduced to observe their effect on recommendation outputs. The results indicate that recommendation feeds differ between accounts under controlled conditions, with varying distributions of political content observed across phases. In particular, interaction behaviour and exposure to politically aligned accounts were associated with measurable shifts in content composition, with differing magnitudes observed between accounts. This project demonstrates the feasibility of building a system to analyse algorithmic personalisation in a structured and repeatable manner, while providing insight into how identity signals may influence content exposure on social media platforms.

Acknowledgements

This project would not have been possible without the support of those around me. I am sincerely grateful to my supervisor Brian McGinley for his guidance, enthusiasm, and willingness to challenge my thinking throughout the development of EchoChamber. To my family, thank you for your constant support and encouragement. I would especially like to acknowledge my grandad, who passed away during this project. He never had the opportunity of an education, and this has stayed with me throughout my time in university and in bringing this work to completion. Finally, this project was motivated by a concern about the increasingly polarised nature of online discourse and its influence on the world we live in. I hope this work contributes, in some small way, to a better understanding of the systems shaping what we see and believe.

Contents

1 Introduction **2**
1.1 Background and Context 2
1.2 Algorithmic Feedback Loops and Content Diversity 3
1.3 Problem Statement 3
1.4 Project Overview 4
1.4.1 Main Components 4
1.4.2 How the System Works 5
1.5 Aims and Objectives 6
1.5.1 Aim 6
1.5.2 Research Question 6
1.5.3 Objectives 6
1.5.4 Success Criteria 7
1.6 Relevance 7
1.7 Project Repository 8
1.8 Chapter Outline 8

2 Methodology **10**
2.1 Software Development Methodology 10
2.2 Research Methodology 11
2.3 Experimental Methodology 11
2.4 Software Development Process 12
2.5 Technology and Tool Selection 12
2.6 Testing and Validation 13
2.7 Ethical Considerations 14

3 Technology Review **16**
3.1 Recommender Systems and Algorithmic Personalisation 16
3.2 Data Capture and Web Interaction 18
3.3 Backend Architecture and REST APIs 19
3.4 Data Storage 20
3.5 AI Processing and Analysis 21

3.6 Data Pipeline and System Workflow . . . 22
3.7 Visualisation and User Interface . . . 23
3.8 Cloud Deployment . . . 24
3.9 Standards and Technologies . . . 25

4 System Design 27
4.1 Overview of the System Architecture . . . 27
4.2 Architectural Style and Design Rationale . . . 28
4.3 Component Design . . . 30
4.4 Chrome Extension . . . 30
4.5 FastAPI backend . . . 34
4.5.1 Role of Backend . . . 34
4.5.2 API Design . . . 35
4.5.3 CORS Middleware . . . 36
4.5.4 Authentication System . . . 36
4.5.5 Study and Session Management . . . 38
4.5.6 Capture Ingestion . . . 38
4.5.7 Backend Processing . . . 39
4.5.8 Database Interaction . . . 41
4.5.9 Design Justification . . . 42
4.6 Database Design (MongoDB) . . . 42
4.6.1 Relationships . . . 45
4.6.2 Indexing . . . 45
4.6.3 Justification . . . 45
4.7 AI Processing Pipeline . . . 45
4.7.1 Tweet Extraction . . . 45
4.7.2 Sentiment Classification . . . 46
4.8 Streamlit Frontend . . . 46
4.8.1 Authentication Interface . . . 46
4.8.2 Study Management Interface . . . 47
4.8.3 Visualisation Interface . . . 47
4.8.4 Backend Communication . . . 49
4.8.5 Design Justification . . . 50
4.9 System Workflow . . . 50

5 System Evaluation 52
5.1 Experimental Results and Observations . . . 52
5.1.1 Overview of Collected Data . . . 52
5.1.2 Political Leaning Analysis . . . 53
5.1.3 Supporting Content Analysis . . . 64
5.2 System Performance Evaluation . . . 66

5.2.1 System Testing 68
5.2.2 Limitations of System and Study 69
5.3 Overall Evaluation 71

6 Conclusion **72**
6.0.1 Core Findings 72
6.0.2 System Evaluation 73
6.0.3 Future Work 74

List of Figures

1.1 High-level architecture of EchoChamber. 5

3.1 Comparison of collaborative filtering and content-based filtering (adapted from [1]). 17

4.1 High-level system architecture of EchoChamber. 27
4.2 Chrome extension popup interface. 30
4.3 Login and authentication flow of the Chrome extension. 32
4.4 Dynamic population of dropdown fields in the Chrome extension. . 32
4.5 Session initialisation and start-capture workflow of the Chrome extension. 33
4.6 Injection of the content script and initiation of scrolling via message passing in the Chrome extension. 33
4.7 Interval-based capture and upload loop of the Chrome extension. . . 34
4.8 Sentiment classification pipeline for extracted tweets. 40
4.9 Sentiment classification pipeline of stored tweets. 41
4.10 Structure of a stored tweet document in MongoDB, showing content, metadata, and analytical fields. 43
4.11 Streamlit frontend displaying topic analysis and study configuration controls. 48
4.12 Streamlit frontend displaying phase calculator. 49
4.13 End-to-end workflow of the EchoChamber system. 51

5.1 Political leaning distribution for male and female accounts in Phase 1. 54
5.2 Political leaning distribution for male and female accounts in Phase 2. 55
5.3 Political leaning distribution for male and female accounts in Phase 3. 57
5.4 Political leaning distribution for male and female accounts in Phase 4. 58

5.5 Tweet liked by both accounts during Phase 5. 60
5.6 Political leaning distribution for male and female accounts in Phase 5. 60
5.7 Political leaning distribution for male and female accounts in Phase 6. 62
5.8 Political leaning distribution for male and female accounts in Phase 7. 63
5.9 Topic distribution for male (left) and female (right) accounts in Phase 2. 64
5.10 Topic distribution for male (left) and female (right) accounts in Phase 6. 65
5.11 Topic by leaning distribution for male (left) and female (right) accounts in Phase 7. 66

List of Tables

4.1 Key FastAPI Endpoints 35
4.2 Overview of MongoDB collections 44

5.1 Accounts followed in Phase 6 61
5.2 Accounts followed in Phase 7 62

Chapter 1

Introduction

1.1 Background and Context

Social media platforms increasingly rely on algorithmic content personalisation to determine which posts, topics, and viewpoints are presented to users. These recommendation systems play a central role in shaping how individuals consume information online, prioritising content based on inferred interests, past interactions, and engagement patterns. Rather than presenting content in chronological order, platforms use complex ranking mechanisms to decide what is most relevant for each user, influencing both the visibility and prominence of different types of content [2]. X (formerly Twitter) open-sourced its recommendation algorithm in March 2023, providing insight into how content is selected and ranked within its personalised feeds. The system combines multiple components, including candidate sourcing from network and non-network content, relationship modelling through Real Graph, interest matching via SimClusters, neural network-based ranking using predicted engagement, and post-ranking heuristics that determine the final feed composition [3]. Prior research has shown that such recommendation systems can significantly influence user exposure by amplifying certain types of content over others [4]. In the case of X, audit-based studies suggest that amplification is driven by engagement signals and community proximity, meaning that users are often exposed to content that is not simply the most recent, but the most algorithmically promoted [4]. These dynamics are important because they can shape the political tone, emotional framing, and diversity of information presented within personalised feeds.

1.2 Algorithmic Feedback Loops and Content Diversity

Algorithmic content personalisation plays a critical role in shaping how recommendation systems evolve over time. A central mechanism underlying this process is the algorithmic feedback loop [2]. Recommendation systems are typically trained on historical user interaction data, such as likes, shares, comments, and watch time. However, these interactions are themselves influenced by prior algorithmic curation, meaning that the data used to update and refine the system is inherently biased rather than neutral [2]. As content receives higher levels of engagement, it is increasingly prioritised within recommendation feeds, leading to greater visibility and a higher likelihood of further interaction. Conversely, content that attracts less engagement is progressively deprioritised, reducing its exposure. This dynamic creates a self-reinforcing cycle in which popular content continues to gain prominence, while alternative or less visible viewpoints are marginalised. Such feedback loops contribute to what is often described as a "rich-get-richer" effect, where dominant narratives become increasingly amplified over time. Chaney et al. demonstrate that these dynamics can distort user behaviour and lead to unequal exposure across content types [5]. Within social media environments, this process may favour engagement-optimised content at the expense of diversity, contributing to the formation of filter bubbles and echo chambers that reinforce existing beliefs and limit exposure to opposing perspectives. These dynamics raise important questions about how recommendation systems shape the type of content users are exposed to, and whether different users may experience systematically different information environments.

1.3 Problem Statement

Recent public discussion has raised concerns about the role of social media platforms in amplifying politically polarised and, in some cases, hostile content. In particular, following the acquisition of X in 2023, some commentators have suggested that the platform's content environment has shifted, with increased visibility of certain political viewpoints. While such claims remain debated, they highlight the importance of understanding how recommendation systems shape user exposure. In addition, personal observation suggests that recommendation feeds and comment sections may differ between male and female users, even where similar content is engaged with. This raises the possibility that identity-related signals may influence how content is curated and presented. Given these uncertainties, there is a need for tools that allow the systematic investigation of how recommendation algorithms behave under controlled conditions. This project therefore focuses

on the design and development of a platform capable of capturing, processing, and analysing personalised content feeds in order to explore whether differences in political content composition can be observed across controlled account variations.

1.4 Project Overview

This project explores how personalised recommendation feeds may differ across accounts with varying identity-related signals through the development of a multi-account research platform. The resulting system, EchoChamber, is designed to capture, store, and analyse content from X's "For You" timeline over time, enabling controlled comparison of content exposure between accounts. Rather than making definitive claims about platform behaviour, the system provides a structured and automated approach for investigating how recommendation outputs may vary under controlled experimental conditions.

1.4.1 Main Components

As shown in Figure 1.1, the main components of the EchoChamber system include a Chrome extension for data capture, a FastAPI backend for processing, MongoDB for data storage, and a Streamlit frontend for data visualisation and study configuration. The system is deployed across cloud platforms, with the FastAPI backend hosted on Render and the Streamlit interface deployed on Streamlit Community Cloud.

The Chrome extension operates on the X platform, where the user logs in and navigates the recommendation feed. After authentication, the user selects the study, subject, and phase to which captured data should be assigned. The extension captures screenshots of the feed and sends them via a REST API to the FastAPI backend.

FastAPI acts as the central processing layer of the system, receiving captured data, processing the collected posts, and storing structured outputs in MongoDB. The backend also integrates with external services, including OpenAI for content extraction and classification, and an Brevo email service for account-related functionality. The backend is hosted on Railway deployment cloud platform.

The Streamlit frontend retrieves processed data from the backend and presents it through visualisations, enabling comparative analysis across different phases of the study. It also provides functionality for creating and managing studies, subjects, and phases, while being hosted on the Streamlit Community Cloud.

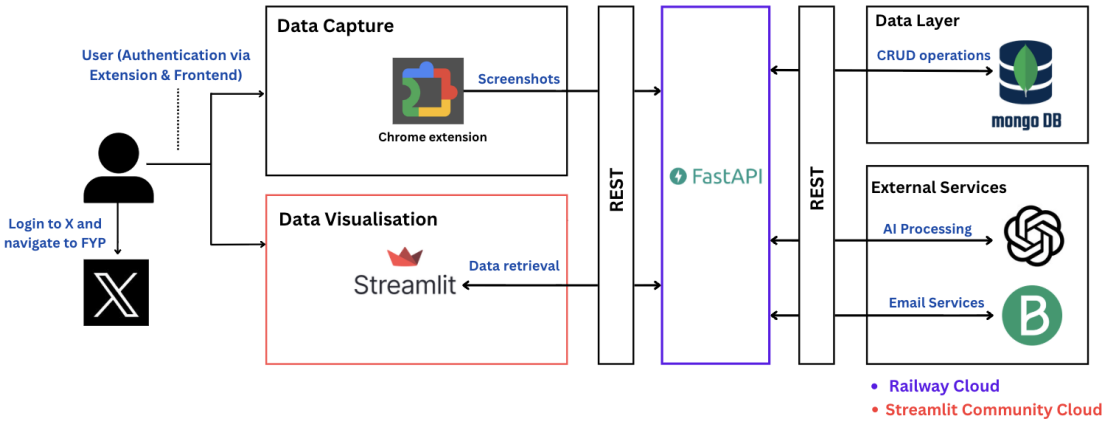

Figure 1.1: High-level architecture of EchoChamber.

1.4.2 How the System Works

At a high level, the system operates as a multi-stage pipeline for capturing, processing, storing, and analysing personalised recommendation feed data from X. The workflow begins with the user interacting with both the frontend and the Chrome extension.

Within the Streamlit frontend, the user can create, update, and delete studies, as well as view and analyse trends in the collected data. To collect new data, the user logs into the Chrome extension using token-based authentication, selects the relevant study, subject, and phase, and initiates the capture process.

The extension then automatically scrolls through the X homepage while capturing screenshots of the recommendation feed. These screenshots are transmitted to the FastAPI backend via a REST API, where they are processed to extract and structure relevant post information.

The backend integrates with external services to support this processing, including OpenAI for content extraction and classification, before storing the resulting structured data in MongoDB. This database serves as the central storage layer for captured posts, study metadata, and analytical outputs.

The processed data is then made available to the Streamlit frontend, where it is retrieved and visualised. Through this workflow, the platform enables comparative analysis of recommendation feeds across controlled accounts, allowing patterns in political leaning and content exposure to be examined over time.

1.5 Aims and Objectives

1.5.1 Aim

The aim of this project was to design and implement a platform capable of supporting longitudinal analysis of social media recommendation behaviour, specifically with regard to the moral stance and political leaning of posts. The system integrates automated capture workflows, backend processing, structured data storage, and analytical visualisation to enable comparison of content shown to multiple accounts under controlled conditions.

Rather than establishing definitive claims about platform behaviour, the project focuses on providing a structured and reproducible framework through which potential differences in recommended content can be explored. In this study, the platform is applied to a controlled case involving two accounts that differ only in gender-related signals, allowing for the examination of whether observable differences in content composition emerge under these conditions.

Given the limited scope of the study, findings are not intended to represent ground truth regarding gender bias within the X recommendation algorithm. Instead, the study serves as a proof of concept, demonstrating how such a system can be used to explore whether identity-related signals, such as gender, may contribute to variations in recommended content, and to observe how different accounts respond to politically oriented material.

1.5.2 Research Question

This dissertation explores how a controlled, system-driven approach can be used to analyse variation in personalised content recommendations on X, with particular attention to political leaning.

The research question addressed in this study is: *To what extent is it possible to design and implement a system capable of detecting potential political bias in the X recommendation algorithm under controlled experimental conditions?*

1.5.3 Objectives

To achieve this aim and address the research question, the project had the following objectives:

- Develop a system to automatically capture content from the X "For You" timeline
- Process and store captured data in a structured format

- Classify and analyse content by political leaning and moral stance
- Provide visualisations for comparison across accounts
- Explore whether observable differences in recommended content emerge between accounts with different gender-related signals

1.5.4 Success Criteria

Success in this project is measured by the extent to which the platform can reliably support the full process of longitudinal recommendation feed analysis. This includes the successful capture of content from X's "For You" timeline, the accurate processing and structured storage of captured data, and the ability to associate this data with the correct study, subject, and phase. Success is also determined by whether the system can generate meaningful comparative insights across controlled accounts over time, particularly in relation to political leaning, moral stance, and patterns of content exposure. In addition, the platform must provide a usable interface for study management and data visualisation, allowing users to create and manage studies while clearly observing trends within the collected dataset. More broadly, the project is considered successful if it demonstrates that a working technical framework can be built to study algorithmic content personalisation in a structured, repeatable, and analytically useful way.

1.6 Relevance

This project is relevant in the context of increasing reliance on algorithmically curated recommendation feeds, such as X's "For You" timeline, which play a central role in shaping how users encounter information online. While prior research has examined the effects of recommendation systems on content exposure[4][6], there remains a need for practical, system-based approaches that allow these effects to be observed under controlled conditions.

This dissertation addresses this gap by developing a working platform that enables the longitudinal capture and analysis of recommendation feed content across multiple controlled accounts. By incorporating identity-related signals, such as gender, within a controlled experimental setup, the study demonstrates how such a system can be used to explore whether observable differences in content composition emerge between accounts under consistent conditions.

The relevance of this work extends beyond technical implementation. Understanding how recommendation systems distribute politically and socially framed content is important for broader discussions around algorithmic transparency, fairness, and digital media literacy. As personalised feeds increasingly shape public

discourse, tools and methodologies that allow for their systematic analysis are of growing importance.

1.7 Project Repository

The source code for this project is available at: `https://github.com/trinv1/EchoChamber`, a recorded demo of the working application is also available at: `https://atlantictu-my.sharepoint.com/:v:/g/personal/g00405393_atu_ie/IQALtKPAkfeHQ4vLcEPlbf0YAdvKnTKDz0FeGVYa3FXMZ8I?e=RYYv2e`

The repository contains the main components of the EchoChamber platform, including:

- the Chrome extension used to capture content from X's "For You" timeline,
- the FastAPI backend responsible for receiving, processing, and storing captured data,
- the MongoDB integration used for structured data storage,
- the Streamlit frontend used for authentication, study management, phase management, subject management, and data visualisation,
- the analysis and charting logic used to display political leaning, topic, and word-frequency results,
- the study protocol used to ensure consistency and control throughout the experimental process, and
- supporting scripts and configuration files required for deployment and execution.

1.8 Chapter Outline

This dissertation is organised into six chapters:

- Chapter 1 introduces the background and context of the study, outlines the project overview, defines the aims, research question, and objectives, and discusses the relevance of the work.
- Chapter 2 presents the methodology adopted for the project, including the overall development and research approach used to design and evaluate the system.

- Chapter 3 provides a review of the key technologies, concepts, and relevant literature, including studies on recommender systems and algorithmic personalisation, that informed the implementation of the platform.
- Chapter 4 describes the system design, outlining the architecture of the application and explaining how its main components interact.
- Chapter 5 evaluates the completed system, examining its performance, functionality, and suitability for supporting comparative analysis of recommendation feed data. It also presents the results of the longitudinal study conducted on the X platform and discusses limitations encountered during development.
- Chapter 6 concludes the dissertation by summarising the key findings of the study, evaluating the effectiveness of the system, and discussing opportunities for future work.

Chapter 2

Methodology

This chapter outlines the methodology adopted for the design, implementation, and evaluation of EchoChamber. Because the project combined software development with the investigation of algorithmic content personalisation, a software development methodology, research methodology and an experimental methodology were required. This chapter therefore explains how the application was built, how the study was structured, and why the overall approach was suitable for addressing the project aims and objectives.

2.1 Software Development Methodology

During the development of this project, an incremental, iterative, and agile-style approach was adopted. Development was organised using GitHub Issues and a GitHub Projects Kanban board, where tasks were first defined and planned before implementation began. Once a task was ready to be developed, it progressed through the stages of the Kanban board until completion. This approach supported structured planning, clear task management, and continuous refinement of the system as new requirements and challenges emerged throughout development.

This approach was particularly well suited to the project because the system was developed in separate stages rather than as a single complete implementation from the outset. In its initial form, the platform was built and tested locally, with feed capture, database propagation, and sentiment processing carried out through scheduled Python scripts. As development progressed, the system evolved into a more integrated cloud-based architecture, where capture, processing, backend services, and frontend interaction were combined into a unified workflow with minimal manual intervention. Because the platform was built progressively and refined over time, an incremental and agile-style approach was appropriate, as it supported continuous improvement, regular testing, and adaptation to new re-

quirements throughout development.

2.2 Research Methodology

The research methodology for this project was grounded in a literature-informed design approach. Initially, relevant academic literature and technical resources were reviewed to inform both the system architecture and the overall research direction. This included studies on algorithmic recommendation systems, audit-based analyses of social media platforms, and research on data pipelines and large-scale data processing. Academic databases such as IEEE Xplore, ACM Digital Library, and Google Scholar were used to identify relevant papers and prior work in these areas.

In addition to academic literature, technical documentation played a key role in the development of the system. Documentation relating to database systems, particularly MongoDB, was used to guide the design of efficient data storage and retrieval structures. As the project involved the development of a Chrome extension, official documentation provided by Google was particularly important in understanding extension architecture, permissions, and communication between components, as prior experience in this area was limited. Furthermore, Railway documentation was used to support deployment and debugging, particularly in relation to interpreting logs and resolving issues within a cloud-hosted environment.

Insights from both academic and technical sources informed key design decisions, including the use of a multi-stage data processing pipeline, structured storage of extracted content, and the adoption of a multi-account framework. This ensured that the system design was grounded in existing research while remaining suitable for practical implementation.

2.3 Experimental Methodology

An experimental methodology was adopted to support the research question. The problem was examined through the use of controlled accounts, where two separate accounts were created and configured to remain identical in all key respects except for assigned gender and gendered usernames. Upon creation, both accounts were given the same topic interests, ensuring no initial differences between them.

The study was structured into a series of phases, with each phase representing a controlled change applied to the accounts. During each phase, data was collected to observe any resulting changes in the recommendation feed. For example, in Phase 1, the accounts operated in a baseline state with no gender assigned, and data was collected over a defined period of time. In Phase 2, each account was

assigned a gender identity within the profile, with one account marked as male and the other as female, after which data collection continued. This process continued through to a seventh and final phase, in which the accounts were deliberately differentiated based on political signals. This phased structure allowed data to be collected longitudinally within clearly defined start and end dates for each phase.

A longitudinal approach to data collection was particularly appropriate, as it allowed recommendation behaviour to be observed as it evolved over time rather than relying on a single snapshot following a phase change. This approach enabled controlled comparison between accounts while limiting the influence of unrelated variables. By changing only one identity-related factor at a time and observing the resulting feed output over time, the methodology provides a structured basis for exploring whether observable differences in recommended content emerge in relation to the applied account signals, while acknowledging that such observations are not sufficient to establish causal relationships or generalisable conclusions.

2.4 Software Development Process

The stages of implementation were as follows:

- Initially, local capture and processing were developed using scheduled Python scripts to support early-stage data collection.
- MongoDB integration was then introduced to provide structured storage for captured and processed data.
- A FastAPI backend was subsequently developed to receive incoming capture data, process collected posts, and store the resulting output in the database.
- Feed capture was later migrated to a Chrome extension, which automated scrolling through X while capturing on-screen content.
- Finally, Streamlit was used to provide a frontend for study management and data visualisation.
- Supporting scripts and configuration files were also developed to enable deployment and system execution.

2.5 Technology and Tool Selection

For feed capture, a Chrome extension written in JavaScript was developed to automatically scroll through a user's X recommendation feed while simultaneously

capturing on-screen content. This approach was selected because direct scraping of X can be unreliable and may lead to bot detection or platform restrictions. Using a browser-based extension allowed feed data to be collected in a way that more closely reflected real user interaction with the platform.

MongoDB was used for data storage because it supports flexible, document-oriented storage through JSON-like structures. This made it well suited to the project, as captured posts, metadata, and analytical outputs could vary in structure over time as the system evolved. Its schema-flexible design also supported incremental development and rapid adaptation during implementation.

FastAPI was used as the RESTful layer of the platform because it is a high-performance Python framework and integrates well with JSON-based APIs. As Python was already used throughout the rest of the application, FastAPI provided a consistent and efficient way to handle incoming capture data, backend processing, and communication between system components. Its compatibility with OpenAPI standards also made API structure and testing more manageable.

Railway was selected to host the backend because it provides a simple cloud deployment platform for Python web services. It allowed the FastAPI backend to be deployed and maintained online without requiring complex infrastructure setup, making it suitable for a student project with evolving requirements.

OpenAI was used for text extraction and sentiment analysis because the project required captured feed content to be transformed into structured textual data and then analysed according to dimensions such as political leaning, moral stance, and emotional tone. Its models provided a practical way to process captured screenshots and generate structured analytical outputs that could then be stored and compared over time.

Streamlit was used for frontend visualisation and study configuration because it provides a simple way to build data-focused web interfaces in Python. It was particularly suitable for this project as it supports rapid dashboard development, interactive visualisation, and straightforward integration with backend outputs. In addition, Streamlit Community Cloud provided a convenient way to deploy the frontend application online.

2.6 Testing and Validation

Testing and validation were carried out throughout development to ensure that each component of the system operated correctly both individually and as part of the wider platform. Because EchoChamber consists of multiple connected components, including the Chrome extension, FastAPI backend, MongoDB database, processing pipeline, and Streamlit frontend, validation was not limited to one stage of development but instead took place continuously as the system evolved.

At the data capture stage, testing focused on whether the Chrome extension could successfully authenticate the user, begin the capture process, scroll through X's "For You" feed, and send capture data to the backend without failure. This was validated by checking that screenshots were captured at the expected intervals and that associated metadata, such as the selected study, subject, and phase, was correctly attached to each request.

Backend testing focused on the reliability of the FastAPI service in receiving incoming capture data, processing it, and storing the output. This included verifying that API endpoints responded correctly, that uploaded data could be handled without error, and that processing steps produced structured outputs in the expected format. Validation at this stage also involved checking that failures in request handling or processing could be identified and corrected during development.

Database validation was carried out to ensure that captured and processed data were stored correctly in MongoDB. This involved checking that documents were inserted into the correct collections, that metadata fields were preserved accurately, and that the stored data could later be retrieved for analysis and visualisation. Particular attention was given to ensuring that posts were associated with the correct study structure so that comparisons between accounts and phases remained valid.

Testing was also applied to the analytical components of the system. This included validating whether extracted text and classification outputs, such as political leaning, moral stance, and emotional tone, were returned in a usable structured format. While these outputs are inherently interpretive, validation focused on whether the system consistently produced data that was sufficiently structured and coherent for comparative analysis.

Finally, end-to-end validation was used to confirm that the full workflow operated as intended. This involved checking that feed data could move successfully from capture, to backend processing, to database storage, and finally to frontend visualisation. The project was therefore validated not only through testing of individual components, but also through confirmation that the complete platform could support the intended process of longitudinal recommendation feed analysis in a reliable and repeatable way.

2.7 Ethical Considerations

This project involved the collection and analysis of content displayed within personalised X recommendation feeds. Although the data was collected from accounts controlled by the researcher, ethical considerations were important because the captured feed could contain posts created by real users. For this reason, the

project focused on aggregate patterns such as political leaning, topic distribution, and content exposure rather than identifying or profiling individual users.

The Chrome extension was designed to capture the recommendation feed as it appeared during normal browser use. It did not attempt to access private data, bypass authentication controls, or interact directly with X's internal systems. Instead, it captured visible on-screen content from accounts created for the purpose of the study. The use of automated scrolling was intended to support consistent data collection across sessions, while avoiding excessive or disruptive request behaviour.

To reduce privacy risks, the analysis presented in this dissertation is based on summarised and aggregated outputs rather than exposing individual user identities. The study therefore treats the collected posts as observational data for examining recommendation behaviour, while recognising that platform terms of service, user privacy, and responsible data handling remain important limitations of this type of research.

The findings are also interpreted cautiously. The study does not claim to reverse-engineer X's recommendation algorithm or make definitive claims about platform-wide bias. Instead, the system is presented as a proof-of-concept tool for controlled observation of personalised feed outputs under defined experimental conditions.

Chapter 3

Technology Review

This chapter reviews the technologies and concepts that underpin the development of the EchoChamber platform. The project investigates algorithmic content personalisation within social media recommendation systems, requiring a combination of data capture, processing, storage, and analysis technologies.

Rather than providing a purely descriptive overview, this chapter examines these technologies at a conceptual level and justifies their selection in relation to the project's aims and methodology. In doing so, it demonstrates how the system design is grounded in established approaches and supports the longitudinal analysis of personalised recommendation feeds.

3.1 Recommender Systems and Algorithmic Personalisation

Recommendation systems have evolved significantly since their emergence in the 1990s. Early systems were primarily based on relatively simple filtering techniques, whereas modern platforms increasingly rely on large-scale machine learning and deep learning approaches to deliver highly personalised content.

Traditional recommendation systems are commonly categorised into content-based filtering, collaborative filtering, and hybrid methods. As shown in Figure 3.1, content-based filtering recommends items by analysing the characteristics of content with which a user has previously engaged and identifying similar items. In contrast, collaborative filtering predicts user preferences by identifying behavioural similarities between users or relationships between items. Hybrid approaches combine elements of both methods in order to improve recommendation accuracy while reducing the limitations associated with relying on a single technique [6]. More recent systems incorporate deep learning techniques, such as neural collaborative filtering and sequence-aware models, enabling the capture of complex user–item

relationships and temporal interaction patterns [7][8].

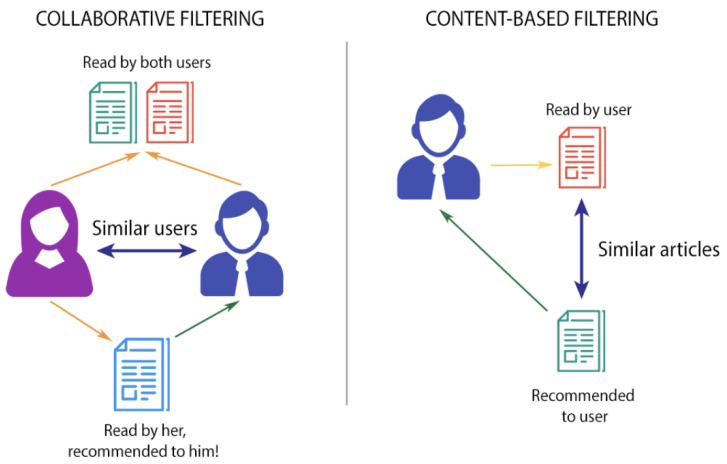

Figure 3.1: Comparison of collaborative filtering and content-based filtering (adapted from [1]).

While these traditional and deep learning-based approaches provide the foundation for recommendation systems, modern social media recommendation systems are typically implemented as multi-stage pipelines rather than single ranking models. In these systems, a candidate generation stage is first used to retrieve a large pool of potentially relevant content, followed by a ranking stage in which machine learning models score and order items based on predicted engagement signals such as likes, replies, and viewing time. Finally, post-ranking heuristics and filtering steps are applied to refine the composition of the feed before it is presented to the user [3, 6].

In the case of X's "For You" timeline, publicly available descriptions indicate that recommendation is driven by candidate sourcing from both network and non-network content, neural network-based ranking, and post-processing heuristics that determine the final ordering of content [3]. This multi-stage approach enables highly personalised content delivery at scale, but also introduces the potential for amplification effects, where certain types of content are disproportionately promoted based on engagement signals.

While these architectural descriptions explain how recommendation systems are designed, empirical studies have examined how such systems behave in practice. Early work suggested that collaborative filtering and popularity-based approaches could lead to selective exposure and reduced content diversity, although these effects were relatively limited in simpler systems [9].

More recent large-scale studies have identified stronger narrowing effects in modern, engagement-optimised systems. For example, audit-based research exam-

ining YouTube's recommendation system found that as users followed successive recommendations, the ideological diversity of content decreased, with recommendations increasingly favouring ideologically aligned or extreme material [10]. Similarly, studies of short-video platforms have demonstrated that content exposure can become progressively narrower as user interaction depth increases, indicating the presence of reinforcing feedback loops within recommendation pipelines [11]. The study also reported that certain demographic characteristics, including gender, were linked to differing levels of exposure concentration, with male users in particular showing a higher tendency towards narrower content distributions. This observation is relevant to the present study, as it motivates the exploration of how gender-related account signals may relate to variation in recommendation behaviour across platforms. These findings suggest that modern recommendation systems not only personalise content but may also shape the range of information users are exposed to, potentially reinforcing existing preferences and limiting exposure to diverse viewpoints.

These characteristics are particularly important in the context of this project, as EchoChamber captures and analyses the outputs of such recommendation pipelines over time, rather than attempting to reconstruct the underlying models directly. By adopting an observational approach, the system enables the exploration of how algorithmic personalisation may relate to content diversity and political exposure across different experimental conditions, without making causal or generalisable claims.

3.2 Data Capture and Web Interaction

Web data capture refers to the process of collecting information from web-based interfaces for subsequent processing, storage, and analysis. In the context of modern social media platforms, this presents particular challenges due to dynamic content rendering, personalised interfaces, and anti-automation mechanisms designed to restrict scraping and other forms of automated interaction. These challenges are especially relevant in this project, as the system was required to capture content from X's personalised "For You" feed in a way that preserved realistic user interaction and reduced the likelihood of platform detection.

Traditional approaches to web data extraction include direct HTTP-based scraping using libraries such as `Requests` and `BeautifulSoup`, alongside browser automation frameworks such as `Playwright` and `Selenium`. While HTTP-based scraping can be effective for static content, modern social media platforms often rely on dynamic rendering and interaction-driven loading, making browser automation more suitable in many cases. API-based access may also be used where available, although this is often limited or restricted.

During the early stages of the project, browser automation using `Playwright` was considered as a potential method for feed capture. However, this approach proved unsuitable due to platform restrictions, anti-bot detection mechanisms, and the difficulty of reliably capturing a personalised feed in a manner consistent with genuine user interaction.

Capturing data from X's recommendation feed presents several challenges. Direct scraping is not a reliable method of collection, as automated scraping behaviour may be detected by the platform as bot activity, potentially resulting in account restrictions or bans. Although API-based access can provide structured access to some platform data, the algorithmically curated "For You" feed is not available through the X API [12]. Consequently, API-based methods are insufficient for capturing the personalised recommendation outputs required for this study.

To address these limitations and support the project objective as effectively as possible, a Chrome extension was developed for feed capture. The implementation of this approach was informed by official Chrome extension development documentation [13]. Operating within the browser environment, the extension was designed to emulate realistic user interaction. When activated, it performs non-uniform scrolling behaviour by varying its speed and introducing occasional pauses in order to better reflect natural browsing patterns. This helped reduce the likelihood of repeated, uniform actions being identified as automated behaviour by the platform's anti-bot mechanisms.

At the same time, the extension captures periodic screenshots of the visible browser tab as rendered on screen, rather than directly interacting with X's internal systems. This made it possible to collect feed content in a manner more consistent with normal platform use while reducing the risk of triggering server-side detection mechanisms.

This approach to data capture was well suited to the longitudinal study of the recommendation feed, where consistency and reliability were essential for collecting meaningful data over time. By capturing the feed as it appeared during each session, the system enabled realistic observation of recommended content and supported detailed evaluation of the posts shown to the user. This also helped preserve the integrity of the experiment, as the extension operated throughout the study without introducing alternative collection methods that could have altered the natural presentation of the feed.

3.3 Backend Architecture and REST APIs

Backend architecture refers to the server-side components of a system responsible for processing data, executing application logic, and managing communication

between different parts of the platform. In a typical client–server model, the frontend (client) interacts with the backend through defined interfaces, allowing data to be transmitted, processed, and returned as structured responses. This separation of concerns enables systems to handle complex operations efficiently while maintaining scalability and modularity.

Representational State Transfer (REST) is a widely adopted architectural style used in the design of web-based APIs. It enables communication between client and server through standard HTTP methods such as GET, POST, PUT, and DELETE. In practice, RESTful APIs typically exchange data using structured formats such as JSON, allowing information to be transmitted and processed in a consistent and platform-independent way across different systems. Due to their stateless nature and scalability, REST APIs are commonly used in modern web applications to support reliable and efficient data exchange between system components.

FastAPI was selected as the RESTful framework for this project due to its high performance and strong compatibility with Python [14]. As the wider system was developed in Python, FastAPI provided a consistent and efficient approach to handling API requests, processing incoming data, and enabling communication between system components.

In addition, FastAPI supports the automatic generation of API documentation through OpenAPI standards, which proved particularly useful during development. This functionality allowed for easy verification that data was being transmitted correctly through the system pipeline, from capture to storage, while also simplifying testing by providing clear visibility of available endpoints and data structures.

The backend of the system received captured data from the Chrome extension, processed incoming requests, and managed communication between the main components of the platform. Within this project, it was responsible for handling uploaded feed captures and associated metadata, coordinating storage in the database, and serving structured data to the frontend for study management and analytical visualisation.

This backend architecture was essential to the overall operation of the platform, as it enabled the automated flow of data from capture through to storage and analysis. By providing a consistent interface between the extension, database, and frontend, it supported the structured and repeatable collection of recommendation feed data required for longitudinal study.

3.4 Data Storage

Data storage refers to the way data is organised and kept so that it can later be retrieved, processed, and analysed. In modern applications, data can be stored in

structured formats, such as relational databases, or in more flexible semi-structured and unstructured formats, depending on the type of information being collected. The choice of storage method therefore depends on the nature, variability, and amount of data within a system.

NoSQL databases are designed to handle data that does not always fit neatly into fixed table structures. Unlike traditional relational databases, they provide more flexible schema designs, allowing fields and document structures to vary over time. This makes them well suited to applications where the format of stored data may change during development or where different types of related information need to be stored together.

MongoDB was selected as the database technology for this project because it provides a document-based NoSQL approach that suited the nature of the collected data[15]. Since the system stored tweet content, screenshots, analytical outputs, and study metadata, the structure of the data was not always uniform. MongoDB's flexible schema made it possible to store these different forms of information in a consistent way while also allowing the system to adapt as the platform developed.

This approach to data storage was essential for supporting the longitudinal analysis carried out in this project. By storing tweet content alongside study, phase, and session metadata, the system enabled structured comparison of recommendation outputs over time. The flexibility of the database also allowed analytical results, such as sentiment classifications, to be integrated directly into each document, supporting efficient retrieval and evaluation of the collected data.

3.5 AI Processing and Analysis

Artificial intelligence (AI) and natural language processing (NLP) techniques are widely used to extract structure and meaning from unstructured textual data. These approaches support tasks such as text classification, sentiment analysis, and topic identification, allowing raw text to be transformed into formats that can be more easily analysed and compared [16][17].

In the context of this project, AI-based processing done using the OpenAI API was necessary due to the nature of the collected data. As feed content was captured as images, the system first required a method of extracting textual information before any analysis could take place. In addition, the extracted tweets needed to be classified according to dimensions such as political leaning, moral stance, and emotional tone in order to support meaningful comparison between recommendation feeds.

OpenAI's API was selected for this system because it provides vision capabilities that allow images to be analysed directly, while also supporting structured text outputs that can be constrained to a predefined JSON schema[18]. This made

it a practical solution for transforming captured feed screenshots into structured data suitable for further analysis, while avoiding the limitations associated with direct scraping.

Within the EchoChamber system, the API is used to extract text from images captured by the extension. The extracted text is then analysed to produce structured classifications, including sentiment, political leaning, and topic. These outputs are stored in MongoDB alongside the corresponding tweet content and metadata, enabling later retrieval and comparative analysis.

The use of AI-based processing was central to the aims of this project, as it transformed captured feed content into data that could be systematically stored, analysed, and compared over time. Without this stage, the screenshots collected by the extension would remain largely unstructured and difficult to evaluate at scale. By converting image-based feed captures into structured textual and analytical outputs, the system was able to support meaningful comparison between accounts, phases, and sessions. This was particularly important for the longitudinal design of the study, where consistent classification of recommendation feed content was needed in order to identify patterns in political leaning, emotional tone, and topical emphasis across the dataset.

3.6 Data Pipeline and System Workflow

A data pipeline refers to the sequence of stages through which data passes from initial collection to its final stored and processed form. As data moves through this pipeline, it may be transformed, analysed, and reorganised in order to support its intended use [19]. Data pipelines are important because they enable information to flow consistently and efficiently from source to destination within a system.

Data pipelines typically consist of a series of connected layers through which data moves and is progressively transformed. Koivisto describes these layers as including the data source, collection, persistence, preprocessing, and modelling stages, each contributing to the movement of data from its original form towards analysis-ready outputs [20]. In practice, this layered structure helps ensure that data is gathered, stored, transformed, and analysed in a consistent and organised way across the system.

In the case of EchoChamber, the pipeline begins with data capture, followed by processing, storage, analysis, and finally visualisation. During the capture stage, screenshots of the personalised feed are collected through the Chrome extension. These are then processed using the OpenAI API to extract text and generate structured classifications. As part of the processing stage, basic preprocessing was applied to ensure the quality of the dataset, including duplicate detection using a combination of hashing and similarity-based comparison. This helped

prevent repeated or near-identical content from being stored and analysed. The resulting data is stored in MongoDB alongside study, phase, and session metadata. Analytical outputs can then be retrieved through the backend and presented via the frontend interface, supporting visualisation and comparison of recommendation feed content over time.

For this system, the data pipeline supported the longitudinal study and analysis of the X recommendation feed over time. The repeatable nature of the process enabled structured comparison of recommendation outputs across different sessions and phases, supporting consistent evaluation of content exposure.

3.7 Visualisation and User Interface

Data visualisation is the process of presenting data in visual forms in order to support understanding and interpretation. By transforming structured data into formats such as charts and dashboards, complex patterns and trends can be identified more easily than through text-based data alone. Effective visualisation improves the clarity of the visual message and helps users identify important patterns in the data [21]. As a result, visualisation plays an important role in modern data-driven systems by enabling users to interact with and analyse large datasets more effectively. In the context of this project, a large dataset accumulated as the project progressed, making visual comparison across sessions, phases, and subjects necessary in order to observe changes in recommendation feeds over time. As the sentiment and political leaning of posts were also being analysed, these outputs needed to be visualised so that patterns in content exposure could be more easily identified throughout the study.

Streamlit was selected as the frontend framework for this project due to its simplicity, flexibility, and strong integration with Python-based systems [22]. It supports the rapid development of interactive dashboards and works well with visualisation libraries, making it well suited to displaying analytical outputs generated by the backend.

Within the EchoChamber system, Streamlit was used to provide an interactive interface for both study management and data analysis. Through this frontend, users were able to create, update, and delete studies, subjects, and phases, allowing the structure of the investigation to be managed directly through the application. The interface also enabled users to select particular studies, phases, subjects, and sessions in order to retrieve relevant data from the backend for analysis.

Once this data was returned through the API, Streamlit was used to present it in a visual and accessible form. This included the display of charts and summary statistics showing patterns in classifications such as political leaning and sentiment across different parts of the dataset. By allowing analytical outputs to be filtered

and viewed according to specific study conditions, the frontend supported more direct analysis of recommendation feed behaviour over time. In this way, Streamlit acted as the main user-facing layer of the platform, connecting processed backend data with an interface suitable for interpretation and comparison.

This visualisation layer was essential for enabling meaningful interpretation of the collected data. By providing a clear and interactive interface, the system allowed patterns in recommendation outputs to be identified across different conditions, supporting the overall aim of analysing algorithmic personalisation over time.

3.8 Cloud Deployment

Cloud deployment refers to the hosting of software systems on remotely managed cloud infrastructure, allowing applications and services to be accessed over the internet rather than being run solely on local machines. Odun-Ayo et al. describe cloud computing as a model through which users can access applications, platforms, and infrastructure on a scalable, pay-as-you-use basis, highlighting its importance in modern system architecture [23]. In practice, cloud deployment supports availability, scalability, and easier access to services without requiring direct management of the underlying hardware by the user.

In the context of this project, cloud deployment was necessary to ensure that both the backend and frontend components of the system could be accessed reliably throughout the study. As EchoChamber depended on communication between the Chrome extension, backend API, database, and visualisation interface, these services needed to be available beyond a single local machine. Hosting the platform online therefore supported more consistent data capture, processing, and analysis across different collection sessions. A Platform as a Service (PaaS) model was particularly suitable, as it allowed both components to be deployed without requiring direct management of the underlying infrastructure, enabling focus on application development rather than server maintenance [24].

For the backend, Railway was selected to deploy the FastAPI service [25]. As a Platform as a Service solution, it offered a simple way to host a Python web application without the need to manage infrastructure directly. It integrates with GitHub, enabling automatic deployment when changes are pushed to the backend, as well as manual deployment from selected commits. This made it a suitable choice for deploying the FastAPI backend used in the project.

For the frontend, Streamlit Community Cloud was used to deploy the Streamlit application [26]. As a Platform as a Service solution, it provided a simple way to host the user interface online without requiring direct management of infrastructure. Its close integration with Streamlit applications and GitHub-based de-

ployment made it well suited to hosting the frontend used for study management, data retrieval, and visual analysis within the project. As the Streamlit library was already being used for dashboard visualisation, Streamlit Community Cloud provided a natural and straightforward deployment option, helping to minimise additional complexity.

This cloud-based deployment approach was important to the overall operation of the platform, as it allowed the main components of the system to remain accessible and connected throughout the study. By hosting the backend and frontend online through managed services, the platform could support the continuous flow of data from capture to processing, storage, and visualisation. This helped provide a stable and integrated environment for the longitudinal collection and analysis of recommendation feed data.

3.9 Standards and Technologies

Standards play an important role in software systems by supporting consistency, compatibility, and interoperability between different components. Interoperability enables systems to communicate and exchange data effectively, allowing information to be shared and reused across platforms.

REST is an architectural style for designing web-based APIs, enabling communication between client and server through standard HTTP methods such as GET, POST, PUT, and DELETE. RESTful APIs are stateless, meaning each request contains all the information required for processing, which supports scalability and reliability in distributed systems. In this project, REST was used to enable communication between the Chrome extension, backend, and frontend, allowing data to be transmitted and retrieved in a consistent and structured manner.

JavaScript Object Notation (JSON) is a lightweight data format commonly used for structuring and exchanging data between systems[27]. Its simple, human-readable structure makes it well suited to web-based applications, particularly in combination with RESTful APIs. In this project, JSON was used as the primary format for transmitting data between the frontend and backend, as well as for storing structured information within the database. This ensured consistency in how data was represented and allowed different components of the system to interpret and process information reliably.

OpenAPI is a standard for describing RESTful APIs in a structured and machine-readable format. It defines key information such as available endpoints, parameters, request bodies, and response formats, allowing both developers and systems to understand how an API is intended to be used. According to the OpenAPI Initiative, the OpenAPI Specification provides a consistent way to carry information through the API lifecycle and enables API capabilities to be understood

independently of the programming language in which the service is implemented [28]. In this project, OpenAPI was used through FastAPI's automatic documentation features, which generated clear interactive documentation for the backend. This supported development and testing by making the available endpoints and expected data structures easier to inspect and validate.

The use of these standards was important to the overall integration of the EchoChamber platform, as they supported consistent communication between the system's main components. REST enabled structured interaction between client and server, JSON provided a common format for representing and exchanging data, and OpenAPI helped define and document the behaviour of the backend API. Together, these standards improved the reliability and clarity of communication across the extension, backend, database, and frontend, supporting the wider operation of the system.

Chapter 4

System Design

4.1 Overview of the System Architecture

EchoChamber is a cloud-based, multi-component system designed to support the automated capture, processing, storage, and analysis of personalised recommendation feed data from X. The architecture follows a client–server model, with distinct components responsible for data collection, backend processing, data persistence, and user interaction.

As illustrated in Figure 4.1, the system follows a layered client–server architecture, with the FastAPI backend acting as the central integration layer. User-facing components, including the Streamlit frontend and Chrome extension, communicate with the backend via RESTful HTTP requests.

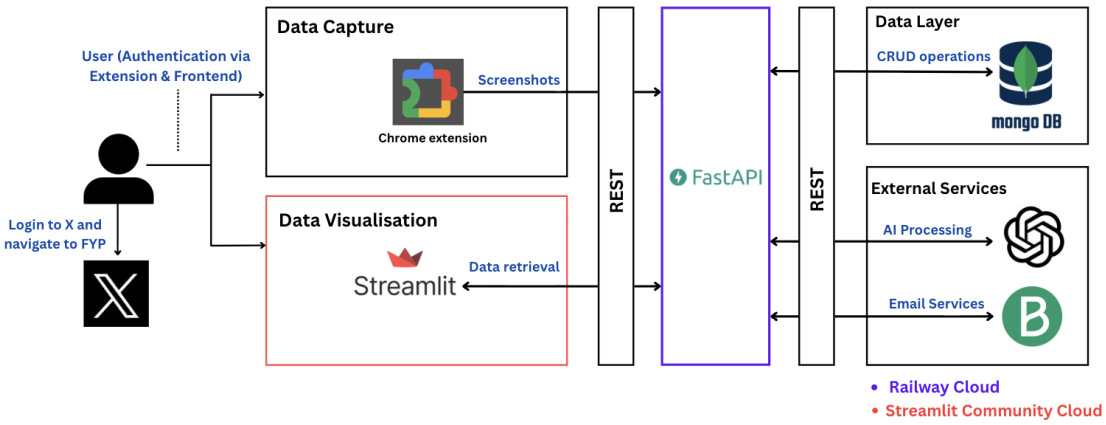

Figure 4.1: High-level system architecture of EchoChamber.

The backend coordinates data storage in MongoDB and integrates with ex-

ternal services such as OpenAI for image processing and Brevo for transactional email delivery. At a high level, the system consists of four primary components: a Chrome extension for automated data capture, a FastAPI backend acting as the central processing layer, a MongoDB database for persistent storage, and a Streamlit-based frontend for user interaction and data visualisation. These components are deployed using cloud-based Platform-as-a-Service (PaaS) solutions, enabling accessibility, scalability, and ease of deployment without direct infrastructure management. These components are described in more detail below.

The Chrome extension operates as the data acquisition layer of the system. It automates interaction with the X platform by simulating user scrolling behaviour and capturing visible content from the "For You" feed. Captured screenshots, along with associated metadata such as timestamps and study identifiers, are transmitted to the backend via HTTP requests.

The FastAPI backend serves as the core of the system, exposing a RESTful API that facilitates communication between all components. It handles user authentication, session management, data ingestion, and processing workflows. The backend is also responsible for integrating external services, including OpenAI for content extraction and analysis, and Brevo for transactional email delivery such as password reset functionality.

MongoDB is used as the primary data store, supporting flexible and scalable storage of heterogeneous data including user records, study configurations, session metadata, captured images, and processed tweet data. Its document-oriented model allows the system to evolve without rigid schema constraints, which is particularly suited to the dynamic nature of captured social media content.

The Streamlit frontend provides a user-facing interface for managing studies, subjects, and phases, as well as for visualising analytical outputs. It communicates exclusively with the backend via HTTP requests, ensuring separation between presentation logic and core system functionality.

Overall, the architecture is designed to be modular and loosely coupled, with each component interacting through well-defined interfaces. This design supports extensibility, allowing individual components such as the data processing pipeline or frontend visualisations to be modified or extended without impacting the overall system.

4.2 Architectural Style and Design Rationale

The EchoChamber system adopts a client–server architectural style, in which client applications request services and resources from a centralised backend server over HTTP. In this model, the server is responsible for handling core application logic, data processing, and persistent storage, while client components focus on user

interaction and data acquisition. This separation enables a clear modular structure, where each component is responsible for a distinct aspect of the system's functionality, improving maintainability and scalability.

Communication between components is implemented using a RESTful API over HTTP, with data exchanged in JSON format using standard web protocols, as discussed in Chapter 3. HTTP methods such as GET, POST, PUT, and DELETE are used to perform operations on server-side resources. This approach ensures a consistent and standardised interface for interaction between system components. The use of RESTful communication promotes loose coupling, as client components, including the Streamlit frontend and Chrome extension, interact with the backend exclusively through defined API endpoints rather than directly accessing the database. The backend therefore acts as a central integration layer, coordinating system operations. This design is reflected in the implementation, where endpoints such as `/login`, `/studies`, `/subjects`, `/phases`, `/sessions/start`, `/sessions/stop`, `/forgot-password`, and `/reset-password` are used to manage system functionality.

The system is deployed using cloud-based Platform-as-a-Service (PaaS) solutions, specifically Railway and Streamlit Community Cloud. Railway is used to host the FastAPI backend, providing automated deployment, environment management, and integration with version control systems such as GitHub. This enables rapid iteration and continuous deployment without requiring manual infrastructure management. The frontend is deployed using Streamlit Community Cloud, which supports direct hosting of Streamlit applications and provides built-in capabilities for interactive data visualisation. This deployment approach ensures that the system is accessible remotely and can operate continuously without reliance on local execution environments.

A document-oriented database model is used through MongoDB to support flexible and scalable data storage. Unlike relational databases, MongoDB does not enforce a rigid schema, allowing the system to store heterogeneous and evolving data structures. This is particularly important in the context of EchoChamber, where captured social media content, metadata, and analytical outputs may vary in structure over time. Collections such as users, studies, subjects, phases, sessions, captures, and tweets are used to organise data logically while maintaining flexibility. This design choice supports agile development and simplifies the integration of new data attributes as the system evolves.

Overall, the architectural design prioritises modularity, scalability, and flexibility. By combining a client–server model, RESTful communication, cloud-based deployment, and a document-oriented database, the system is able to support reliable data collection, processing, and analysis while remaining adaptable to future extensions.

4.3 Component Design

This section outlines the design and implementation of the core components that comprise the FeedScope system, detailing how each part contributes to the overall functionality of the platform.

4.4 Chrome Extension

As shown in Figure 4.2, a Chrome extension was developed for EchoChamber to serve as the primary capture mechanism for collecting content from the X "For You" recommendation feed. The extension operates within the user's live browser session and is responsible for initiating automated feed capture when the user selects the start button in the popup interface. Once activated, it performs human-like scrolling behaviour down the feed while capturing visible content for later processing. This scrolling behaviour includes irregular pauses and dynamic speed variation in order to better resemble natural user interaction.

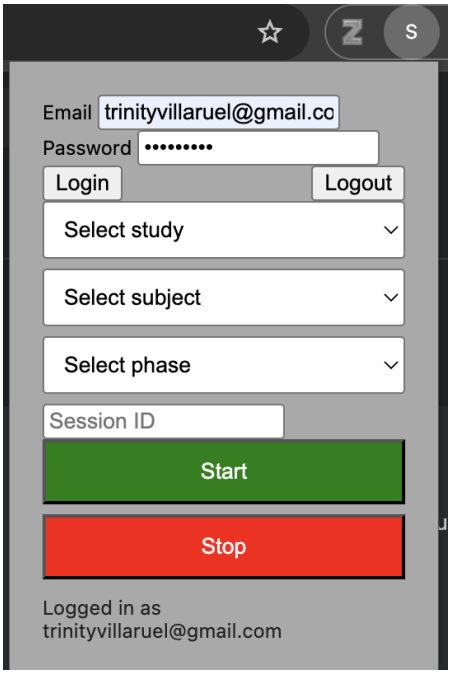

Figure 4.2: Chrome extension popup interface.

Originally, local scripts using Python automation and schedulers were considered to run on a scheduled basis and record data. However, these approaches were rejected due to the risk of detection by X's servers, a lack of realism as the system

would remain tied to a single local machine, and scalability limitations when expanding data collection. The development of a Chrome extension was considered the most appropriate solution, as it integrates directly with real user sessions while mimicking human behaviour, thereby minimising the need for manual intervention.

The extension is built from the following components:

- `manifest.json`: Defines the extension configuration using Manifest V3, including the popup interface, background service worker, and required permissions such as tabs, scripting, storage, downloads, and host access to X and the backend API.
- `popup.html`: Provides the user interface of the extension, including input fields for login credentials, dropdown menus for study configuration, and controls to start and stop the capture process.
- `popup.js`: Handles user interactions within the popup interface. It manages login submission, stores the authentication token, retrieves user-specific data (studies, subjects, and phases), and updates the dropdown fields dynamically.
- `background.js`: Coordinates the capture process. It injects the content script into the active tab, initiates capture sessions, captures visible tab images at regular intervals, attaches metadata, and uploads the captured data to the backend.
- `content.js`: Controls in-page behaviour, including human-like scrolling and detection of the active experimental account. It uses `requestAnimationFrame`, randomised pauses, and variable scrolling speeds to simulate natural browsing behaviour.

Before data collection can begin, the user must authenticate and configure the capture session through the extension popup interface. As shown in Figure 4.3, the user enters their credentials into the popup form defined in `popup.html`, after which the extension handles the submission and sends an authentication request to the backend via a HTTP POST request to the `/login` endpoint.

The backend then validates the credentials and, if authentication is successful, returns an authentication token. This token is stored in the browser extension's local storage and is included in the `Authorization` header of subsequent API requests, enabling authenticated interaction with protected backend endpoints.

As shown in Figure 4.4, once authenticated, the extension retrieves the current user associated with the token and requests the studies available to that user via the `/studies` endpoint.

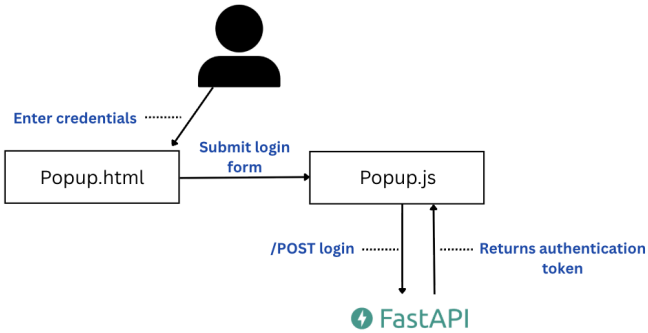

Figure 4.3: Login and authentication flow of the Chrome extension.

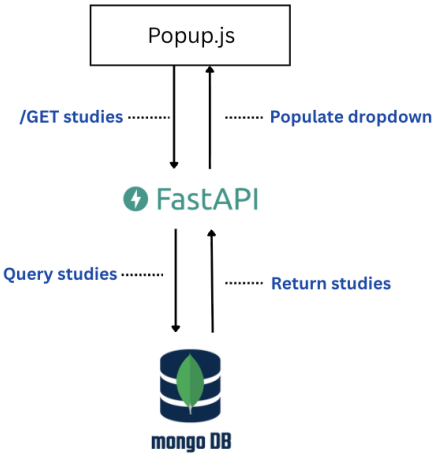

Figure 4.4: Dynamic population of dropdown fields in the Chrome extension.

Figure 4.5 shows that data capture is initiated when the user clicks the start button in the extension popup. This triggers a request from `popup.js` to `background.js`, which coordinates the capture process. The background script sends a POST request to the `/sessions/start` endpoint, including relevant metadata such as the tab ID, study ID, subject ID, and phase ID. The FastAPI backend creates a new session entry in the MongoDB database and returns a session identifier. This session ID is then used to associate subsequent captured data with the correct experimental session.

The background script then injects the content script into the active browser tab, as shown in Figure 4.6. Once loaded, the content script becomes available within the page context. A `START_SCROLL` message is then sent from `background.js` to `content.js`, which initiates simulated scrolling behaviour on the X feed.

Scrolling behaviour was implemented within the content script using browser-

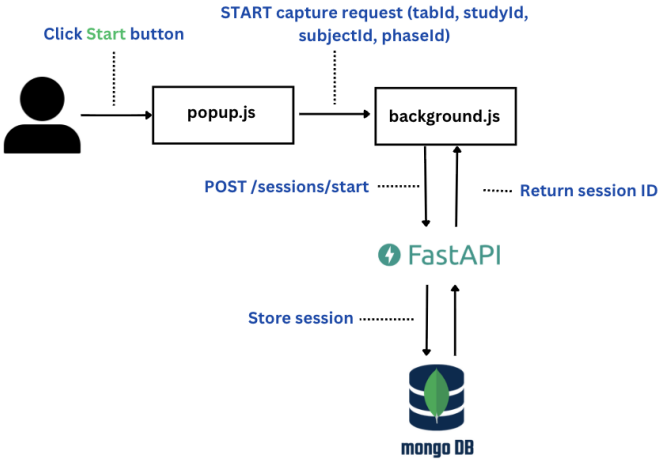

Figure 4.5: Session initialisation and start-capture workflow of the Chrome extension.

based frame updates, allowing the page position to change gradually over time. This supported the introduction of random pauses, variable speed, and tempo changes to better simulate natural browsing behaviour.

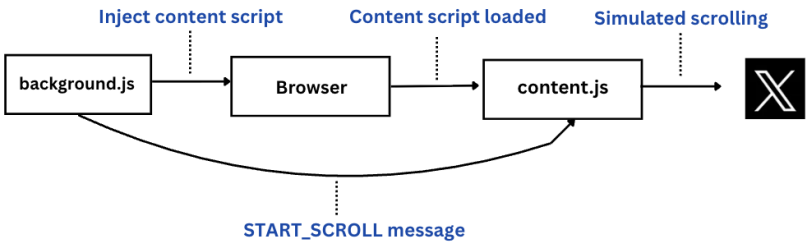

Figure 4.6: Injection of the content script and initiation of scrolling via message passing in the Chrome extension.

As shown in Figure 4.7, once a capture session is active, the background script executes a repeated capture loop at fixed intervals. We see in Listing 4.1, during each iteration the `captureVisibleTab()` function is used to capture the current view of the browser in jpeg format. The captured image is stored in binary, along with associated metadata such as session and study identifiers, is then sent to the backend via a POST request to the `/upload` endpoint. The FastAPI backend processes this request and stores the captured data in the MongoDB database. This loop continues for the duration of the session, enabling continuous data collection from the X feed.

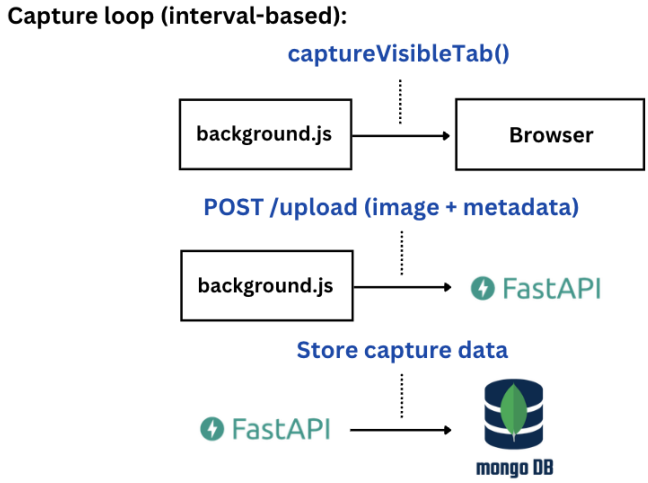

Figure 4.7: Interval-based capture and upload loop of the Chrome extension.

```
const dataUrl = await chrome.tabs.captureVisibleTab(tab.windowId, {
  format: "jpeg",
  quality: 70
});

const result = await uploadToRailway(
  dataUrl,
  capturingTabId,
  tab.url,
  ts,
  studyId,
  subjectId,
  phaseId,
  sessionId
);
```

Listing 4.1: Capture and upload process in `background.js`.

4.5 FastAPI backend

4.5.1 Role of Backend

The FastAPI backend serves as the central coordination and processing layer of the EchoChamber system, providing a RESTful API through which all core compo-

nents communicate. This interaction is illustrated in Figure 1.1 at the beginning of this chapter. It acts as the intermediary between the Chrome extension, Streamlit frontend, MongoDB database, and external services, enabling a modular and decoupled architecture. The backend is responsible for handling user authentication, managing studies and sessions, ingesting captured data from the extension, and coordinating downstream processing tasks such as tweet extraction and sentiment analysis. By centralising application logic within a single service, the backend ensures consistency, scalability, and efficient integration across the system.

4.5.2 API Design

The FastAPI backend exposes a RESTful API that enables communication between the Chrome extension, Streamlit frontend, and the database. All interactions are performed using HTTP requests, with data exchanged in JSON format. This approach allows both client components to operate independently of the database, improving modularity and maintainability.

The API is organised into functional groups, including authentication, study management, session control, and data ingestion. The key endpoints used to support these interactions are summarised in Table 4.1. Authentication endpoints handle user login, verification, and password management, while study-related endpoints provide access to studies, subjects, and phases required for experiment configuration. Session endpoints are used to start and stop capture sessions, ensuring that all collected data is correctly linked to a specific study context. In addition, ingestion endpoints receive captured screenshots and associated metadata from the Chrome extension for further processing.

This structured API design ensures that all components interact through a consistent interface, supporting scalability and simplifying future system extensions.

Table 4.1: Key FastAPI Endpoints

Endpoint	**Method**	**Description**
/signup	POST	Create a new user account
/login	POST	Authenticate user and return token
/studies	GET	Retrieve available studies
/subjects	GET	Retrieve subjects for a study
/phases	GET	Retrieve phases for a study
/sessions/start	POST	Start a capture session
/sessions/stop	POST	Stop a capture session
/upload	POST	Upload captured screenshot and metadata

4.5.3 CORS Middleware

As the EchoChamber system uses separate frontend and backend services, Cross-Origin Resource Sharing (CORS) middleware was added to the FastAPI backend to allow requests from external clients such as the Streamlit frontend and Chrome extension. This was necessary because the frontend and backend are hosted separately, meaning browser-based requests may otherwise be blocked by cross-origin restrictions. The middleware enables the backend to accept HTTP requests from the deployed frontend and extension while still keeping the API structure centralised within FastAPI.

4.5.4 Authentication System

The FastAPI backend implements a secure authentication system to control access to the EchoChamber platform. Users are required to register using an email and password, which are securely stored using hashed representations rather than plain text. As shown in Listing 4.2, the system uses the bcrypt hashing scheme to protect user credentials, and submitted passwords are verified against stored hashes during login.

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

#Hash password on signup
hashed_password = pwd_context.hash(password)

#Verify on login
if not pwd_context.verify(password, user["password_hash"]):
    raise HTTPException(status_code=401, detail="Invalid credentials")
```

Listing 4.2: Password hashing and verification during login.

To ensure that only valid accounts can access the platform, an email verification mechanism is implemented using time-limited tokens. Upon registration, a unique verification token is generated using a secure serializer and embedded within a verification link. This token is later validated by the backend with an enforced expiry period, as shown in Listing 4.3. Only users who successfully verify their email address are allowed to log in, preventing unauthorised or invalid account usage.

The system integrates the Brevo email service to handle transactional email delivery, including account verification and password reset functionality. Verification emails are sent via the Brevo API using HTTP requests, as illustrated in Listing

```
serializer = URLSafeTimedSerializer(SECRET_KEY)

#Generate token
verify_token = serializer.dumps(user_email, salt="email-verify")

#Validate token (1 hour expiry)
token_email = serializer.loads(
    verify_token,
    salt="email-verify",
    max_age=3600
)
```

Listing 4.3: Generation and validation of email verification tokens.

4.4. This integration enables reliable delivery of verification links and highlights the use of external services within the system architecture.

```
response = requests.post(
    "https://api.brevo.com/v3/smtp/email",
    json=payload,
    headers={
        "accept": "application/json",
        "api-key": BREVO_API_KEY,
        "content-type": "application/json"
    }
)

if response.status_code >= 400:
    raise HTTPException(status_code=500, detail="Failed to send verification email")
```

Listing 4.4: Sending verification email using Brevo API.

In addition to verification, the authentication system supports password management through secure reset mechanisms. Time-limited tokens are used to ensure that password reset requests are valid only for a defined period, reducing the risk of misuse. These features contribute to a robust authentication system that balances security and usability.

Overall, the use of password hashing, token-based verification, and external email integration ensures that access to system functionality and study data is restricted to authorised users, demonstrating a practical implementation of secure authentication within the platform.

4.5.5 Study and Session Management

The FastAPI backend provides functionality for managing studies, subjects, phases, and sessions, which together define the structure of the experimental data collected in EchoChamber. A study represents a single experiment, while subjects correspond to individual accounts being analysed, and phases define different stages of the experiment. Sessions are used to represent individual data collection runs within a specific study, subject, and phase.

The backend exposes endpoints that allow both the Chrome extension and Streamlit frontend to retrieve and interact with this structured data. For example, the extension dynamically loads available studies, subjects, and phases to populate user interface controls, ensuring that all captured data is correctly labelled before a session begins. When a capture session is started, a unique session identifier is created and stored, allowing all subsequent captures to be linked to that specific run.

This structured approach ensures that all collected data is organised consistently, enabling accurate comparison between different accounts and experimental conditions. It also supports reproducibility by clearly separating data across studies, phases, and sessions, which is essential for analysing changes in recommendation behaviour over time.

4.5.6 Capture Ingestion

The FastAPI backend is responsible for receiving and storing data captured by the Chrome extension during active sessions. As the extension scrolls through the user's feed, screenshots are taken at regular intervals and transmitted to the backend via HTTP POST requests. Each request includes the captured image along with associated metadata such as the study ID, subject ID, phase ID, session ID, and timestamp.

Upon receipt, the backend reads the uploaded image file and stores it in the MongoDB database as binary data, alongside its corresponding metadata, illustrated in Listing 4.5. Each capture is assigned an initial status indicating that it is queued for processing. This design allows the ingestion process to remain lightweight and responsive, as computationally intensive tasks such as text extraction and sentiment analysis are deferred to a separate background processing stage.

By separating data capture from data processing, the system ensures that the Chrome extension can continue capturing content without interruption, while the backend handles storage and prepares the data for subsequent analysis.

```python
@app.post("/upload")
async def upload_capture(
    image: UploadFile = File(...),
    study_id: str = Form(""),
    session_id: str = Form("")
):
    data = await image.read()
    captures.insert_one({
        "image": Binary(data),
        "study_id": study_id,
        "session_id": session_id,
        "status": "queued"
    })
    return {"ok": True}
```

Listing 4.5: Capture upload endpoint.

4.5.7 Backend Processing

The extraction of text from captured images and subsequent sentiment classification are not performed within a single request. Instead, the processing pipeline is divided into multiple stages to improve efficiency and scalability.

As shown in Figures 4.8, captures are uploaded to the FastAPI backend via HTTP POST requests and stored in MongoDB with a status field set to `"queued"`, indicating that the document is awaiting processing.

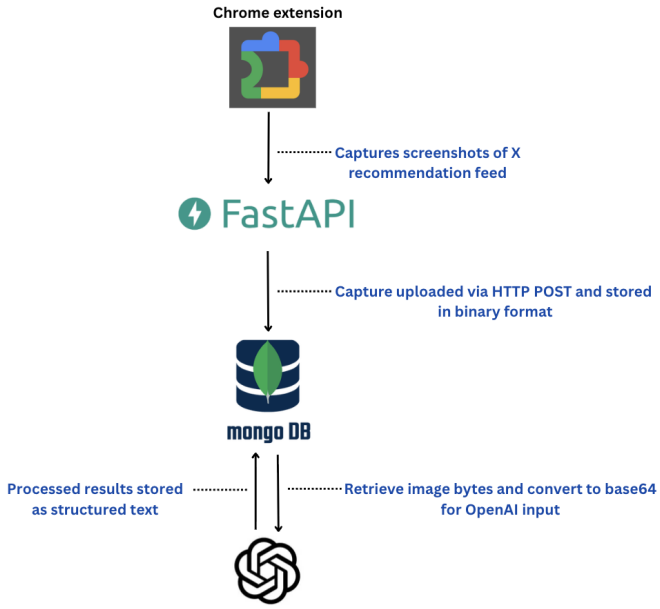

Figure 4.8: Sentiment classification pipeline for extracted tweets.

A background worker runs asynchronously and continuously monitors the database for unprocessed captures. Documents are processed in batches of 10, followed by a short delay. This batching strategy is implemented to respect OpenAI API rate limits, avoiding excessive request usage. For each capture, image data is retrieved from the database and converted into base64 format for processing by the OpenAI API. A structured request is then issued to extract tweet content from the image. As multiple tweets may appear within a single capture, and retweeted content must preserve context, the extraction process is guided by strict formatting rules to ensure consistency and accuracy.

To maintain data quality, duplicate tweets are not stored. Extracted text is normalised and assigned a hash value, which is used to check for existing entries in the database. If a matching hash is found, the tweet is skipped. In addition, similarity scoring is applied to detect near-duplicate content with minor variations. Tweets exceeding a predefined similarity threshold of 94 are also excluded.

Once all queued captures have been processed, the system proceeds to sentiment classification, as shown in Figure 4.9. Tweets that have not yet been analysed are selected and processed in a similar staged manner. This ensures that computationally intensive tasks are handled in the background without affecting the responsiveness of the API.

This staged processing approach ensures that data ingestion remains lightweight while enabling scalable, asynchronous analysis of large volumes of captured data.

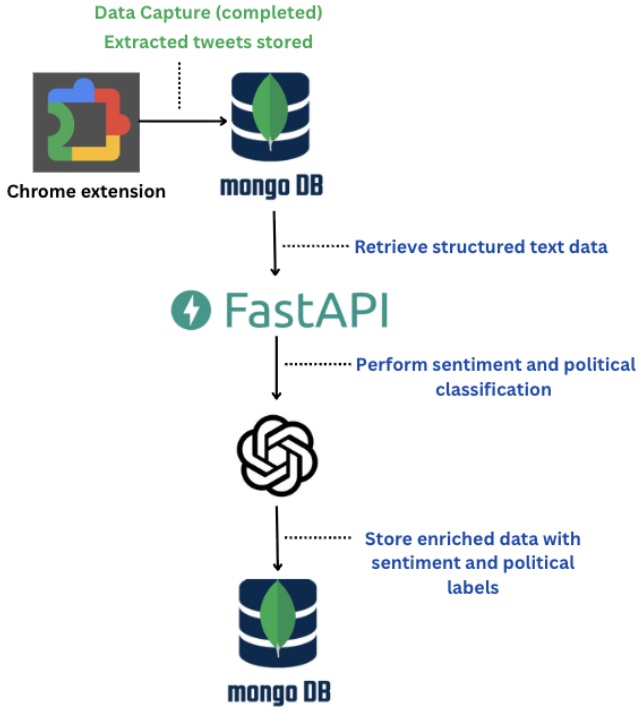

Figure 4.9: Sentiment classification pipeline of stored tweets.

4.5.8 Database Interaction

The FastAPI backend interacts with MongoDB to store and retrieve all application data. Collections are used to represent key entities within the system, including users, studies, subjects, phases, sessions, captures, and tweets. This document-based structure allows flexible storage of both structured metadata and unstructured data such as extracted tweet content and sentiment results.

During capture ingestion, image data and associated metadata are stored in the captures collection, with a status field used to track processing progress. As the background worker processes each capture, extracted tweet data is stored in the tweets collection, where additional fields such as sentiment, topic, and political alignment are later appended.

Indexes are defined on key fields such as user email, study identifiers, and tweet hashes to ensure data integrity and improve query performance. This enables efficient retrieval of data for both processing and visualisation, supporting the overall scalability of the system.

4.5.9 Design Justification

FastAPI was selected as the backend framework because it provides a lightweight and efficient way to develop RESTful APIs in Python. This was well suited to EchoChamber, as the backend needed to coordinate communication between multiple system components, including the Chrome extension, Streamlit frontend, MongoDB, and OpenAI processing pipeline. FastAPI also supports asynchronous programming, which was useful for handling background processing tasks without blocking the main application flow. In addition, its clear route structure and automatic API documentation improved development, testing, and maintainability throughout the project.

4.6 Database Design (MongoDB)

The EchoChamber system uses MongoDB as its primary database for storing and managing application data. MongoDB is a document-oriented NoSQL database, which allows data to be stored in flexible JSON-like structures rather than rigid relational tables. This approach is well suited to the system, as it needs to handle a combination of structured metadata, such as study and session identifiers, and semi-structured data, such as extracted tweets and sentiment analysis results. By using a document-based model, the database can efficiently store and evolve complex data without requiring strict schema definitions.

As shown in Figure 4.10, each document contains the extracted tweet content, along with metadata such as study, subject, phase, and session identifiers. In addition, analytical outputs generated during processing, including sentiment classification, political leaning, and topic, are stored within the same document. This approach allows all relevant information associated with a captured post to be retrieved efficiently in a single query, supporting the longitudinal analysis performed in this study.

As shown in Table 4.2, the database is organised into multiple collections, each representing a key part of the EchoChamber system. The `users` collection stores account information used for authentication and access control. The `studies`, `subjects`, and `phases` collections define the structure of each experiment, allowing captured data to be grouped according to the study being conducted, the account being observed, and the stage of the experiment. The `sessions` collection records individual capture runs, making it possible to distinguish data collected at different times within the same study. Raw screenshots and their associated metadata are stored in the `captures` collection, where each document is marked with a processing status. Once text has been extracted, the resulting tweet data is stored in the tweets collection, where additional fields such as sentiment, political

```
owner_id : "69c530d689c4409cd378c379"
study_id : "1"
subject_id : "1"
phase_id : "5"
session_id : "session_1775658640501"
capture_id : "69d666f69f1ea429cf4d4516"
image_name : "08-04-2026"
username : "elonmusk"
display_name : "Elon Musk"
tweet : "Yes Elon Musk nails exactly why the woke mind virus is fundamentally r…"
post_type : "quote_tweet"
actor_commentary : "Yes"
referenced_username : "XFreeze"
referenced_display_name : "X Freeze"
referenced_post_text : "Elon Musk nails exactly why the woke mind virus is fundamentally racis…"
relationship_to_referenced_post : "endorses"
full_visible_meaning : "Yes Elon Musk nails exactly why the woke mind virus is fundamentally r…"
tweet_normalized : "yes elon musk nails exactly why the woke mind virus is fundamentally r…"
tweet_hash : "3cd24a5f03da039244fdede9f76d9ce823ac8c56ed0bc8b241a5b880780883b6"
likes : ""
retweets : ""
▾ sentiment : Object
  emotional_valence : "negative"
  emotion_intensity : 0.7
  moral_stance : "supportive"
  political_leaning : "right"
  is_toxic : false
  topic : "woke culture criticism"
  stance_reason : "The user, Elon Musk, quotes and endorses the referenced post, which ex…"
```


Figure 4.10: Structure of a stored tweet document in MongoDB, showing content, metadata, and analytical fields.

alignment, and topic can later be added.

Table 4.2: Overview of MongoDB collections

Collection	Purpose
users	Stores account details
studies	Defines experiments
subjects	Represents accounts (boy/girl)
phases	Experiment stages
sessions	Individual runs
captures	Raw screenshots + metadata
tweets	Extracted + processed data

The `captures` collection, shown in Listing 4.6 stores the raw screenshots uploaded by the Chrome extension together with the metadata needed to identify the context of each capture. This includes fields such as study, subject, phase, and session identifiers, along with a timestamp and processing status. The collection acts as the initial storage point in the backend pipeline before tweet extraction takes place.


```
{
  "owner_id": "69c530d689c4409cd378c379",
  "filename": "capture.png",
  "content_type": "image/jpeg",
  "image_bytes": "Binary.createFromBase64('/9j/4AAQSkZJRgABAQAAAQABAAD/4gIoSUND
  "tab_id": "2069388503",
  "page_url": "https://x.com/home",
  "ts": "2026-04-10T20:07:47.536Z",
  "study_id": "1",
  "subject_id": "1",
  "phase_id": "5",
  "session_id": "session_1775851633345",
  "created_at": "2026-04-10T20:07:49.870+00:00",
  "status": "done",
  "processed_at": "2026-04-10T20:11:03.232+00:00"
}
```


Listing 4.6: Example document stored in the captures collection.

4.6.1 Relationships

Collections within the database are linked using shared identifiers such as `study_id`, `subject_id`, `phase_id`, and `session_id`. Rather than using joins as in relational databases, MongoDB follows a document-based approach where related data is connected through these reference fields. This allows for a flexible schema, enabling documents to evolve as additional fields such as sentiment or topic are introduced without requiring structural changes.

4.6.2 Indexing

Indexes are defined on key fields such as user email, study identifiers, and tweet hashes to improve query performance and ensure efficient data retrieval. In addition, unique indexes are used where appropriate to prevent duplicate entries, such as ensuring that each user email or tweet hash is stored only once.

4.6.3 Justification

MongoDB was selected due to its flexible schema, which is well suited to handling semi-structured data such as extracted tweets and sentiment results. Its document-based model allows data to evolve over time, while also providing efficient storage and scalability for large volumes of captured and processed data.

4.7 AI Processing Pipeline

The AI processing pipeline in EchoChamber is responsible for transforming raw screenshot captures into structured analytical data. This is performed in two stages: tweet extraction from captured images, followed by sentiment classification of the extracted tweet content. By separating these tasks, the system is able to convert visual feed data into a format suitable for storage, comparison, and visual analysis.

4.7.1 Tweet Extraction

Once captures are stored in the database in binary form, the data is sent to the OpenAI Vision API for extraction. This enables the system to analyse and interpret screenshot images to identify tweet content.

A structured prompt is used to guide the model, providing context-aware rules to ensure accurate extraction. The model is instructed to distinguish between original tweets and retweets, preserving contextual relationships where necessary. In

addition, it is directed to process only the central feed content, ignoring irrelevant interface elements such as sidebars, navigation menus, and buttons.

Following extraction, the identified tweets and associated data are returned in a structured JSON format, allowing them to be stored and processed consistently within the system.

4.7.2 Sentiment Classification

Once tweet extraction has been completed, the extracted data is then analysed through a sentiment classification stage.

Sentiment classification includes categorising political alignment into *left*, *right*, *centre*, *apolitical*, or *unclear*, while also identifying topic and emotional valence. To reduce the risk of human bias in the classification criteria, the prompt used for sentiment analysis was generated with assistance from ChatGPT and based on commonly accepted political perspectives and definitions. The resulting classifications are returned in structured JSON format and stored back in MongoDB for later analysis and visualisation.

Overall, the AI processing pipeline enables EchoChamber to transform raw screenshot captures into structured analytical data. By separating tweet extraction from sentiment classification, the system is able to process visual feed content in a consistent and scalable manner. The resulting structured outputs support downstream storage, comparison, and visualisation, making it possible to analyse differences in recommended content across accounts and experimental phases.

4.8 Streamlit Frontend

The Streamlit frontend provides the main user-facing interface of the EchoChamber system. It is used to support account access, study configuration, and the visualisation of processed results. Through this interface, users can interact with the platform without directly accessing the database or backend code, allowing the system to present administrative and analytical functionality in a simple and accessible way.

4.8.1 Authentication Interface

The Streamlit frontend provides an interface for user authentication, allowing users to register, log in, and manage their account securely. The interface includes forms for account creation and login, where user credentials are submitted to the FastAPI backend for verification. Additional functionality is provided for password

management, including password reset requests and updates through secure token-based workflows. Email verification is also handled through this interface, ensuring that only authorised users can access the platform. This authentication layer ensures controlled access to study data and system functionality

4.8.2 Study Management Interface

A study management interface is provided for managing studies, subjects, phases, and sessions within the EchoChamber system. Authenticated users can create and organise studies, define subjects representing individual accounts, and configure phases to represent different stages of the experiment. This structured interface ensures that all captured data is correctly labelled and associated with the appropriate experimental context. By managing these elements through the frontend, users can maintain consistency across data collection and support accurate comparison between different accounts and experimental conditions.

4.8.3 Visualisation Interface

As shown in Figure 4.11, the frontend presents study configuration controls alongside visual representations of processed data. Data retrieved from the FastAPI backend is displayed through charts and tables, enabling comparison across studies, subjects, phases, and sessions. Visual outputs such as distributions of political alignment and sentiment trends help identify patterns in the recommended content.

The frontend retrieves processed data through dedicated API endpoints that return aggregated results, such as distributions of political leaning, top words, and topic trends. Within the frontend, helper functions are used to send requests to these endpoints and parse the returned JSON responses. The resulting data is then displayed using pie and horizontal bar charts, allowing users to analyse trends across studies, subjects, phases, and sessions.

Listing 4.7 shows an example of a helper function used within the Streamlit frontend to retrieve aggregated political leaning data from the FastAPI backend. A parameter dictionary is built based on the study, subject, phase, and session selected by the user, then sends a GET request to the appropriate API endpoint. The JSON response returned by the backend is parsed and passed to the visualisation layer of the frontend, where it can be transformed into charts and summary statistics. This illustrates how the frontend remains lightweight, with responsibility focused on user interaction and presentation, while data retrieval and aggregation are handled by the backend.

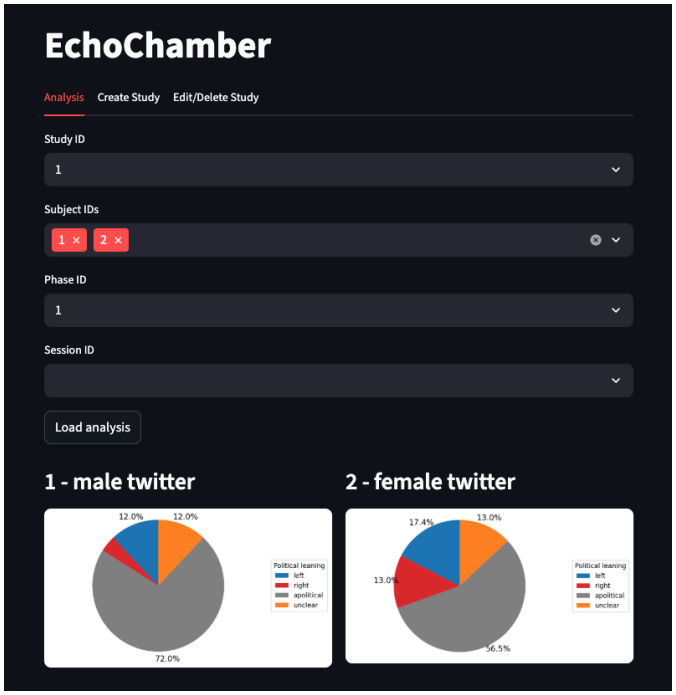

Figure 4.11: Streamlit frontend displaying topic analysis and study configuration controls.

```
def fetch_political_leaning(study_id="", subject_id="", phase_id="", session_id=
    params = {}
    if study_id:
        params["study_id"] = study_id
    if subject_id:
        params["subject_id"] = subject_id
    if phase_id:
        params["phase_id"] = phase_id
    if session_id:
        params["session_id"] = session_id

    r = requests.get(f"{API_BASE}/stats/political-leaning", params=params, heade
    r.raise_for_status()
    return r.json()
```

Listing 4.7: Fetching political leaning data from the FastAPI backend.

In addition to standard visualisations, the interface provides an interactive comparison tool that allows users to calculate differences in political leaning dis-

tributions between selected phases for a given subject, as shown in Figure 4.12. This enables more direct analysis of how recommendation patterns evolve across experimental conditions.

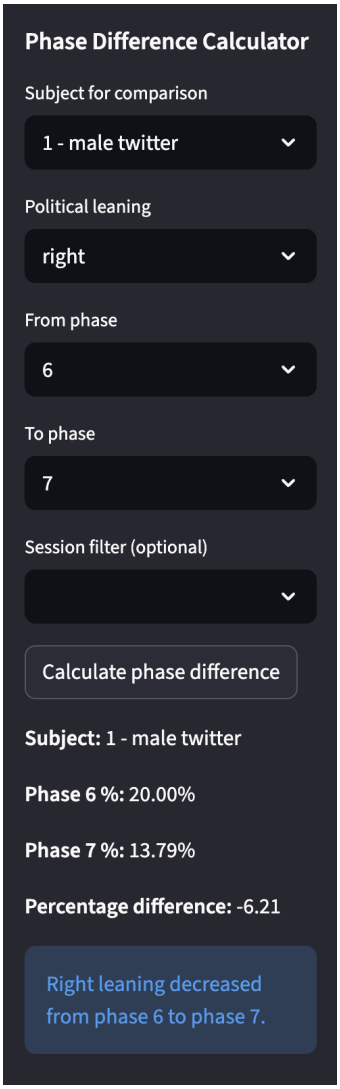

Figure 4.12: Streamlit frontend displaying phase calculator.

4.8.4 Backend Communication

The frontend communicates with the FastAPI backend through HTTP requests, exchanging data in JSON format. All operations, including authentication, study management, and data retrieval, are handled through API endpoints exposed by

the backend. This design ensures that the frontend remains decoupled from direct database access, improving modularity and maintainability while allowing the backend to act as the central point of control.

4.8.5 Design Justification

Streamlit was selected for the frontend due to its simplicity and suitability for rapidly developing data-driven interfaces. It provides built-in support for charts, tables, and interactive components, making it well suited for visualising analytical results. In addition, Streamlit integrates easily with Python-based backends, allowing seamless communication with the FastAPI service and enabling efficient development of the overall system.

4.9 System Workflow

Illustrated in Figure 4.13, the EchoChamber system operates through a structured end-to-end workflow that connects data capture, processing, and visualisation. Users begin by authenticating through the Chrome extension and selecting the relevant study, subject, and phase. A capture session is then initiated, during which the extension collects screenshots of the recommendation feed and sends them to the FastAPI backend.

Captured data is stored in MongoDB and processed asynchronously by a background worker. The processing pipeline extracts tweet content using the OpenAI API, followed by sentiment classification of the extracted data. The resulting structured data is stored in the database and made available for analysis.

Finally, the Streamlit frontend retrieves processed data through backend API endpoints and presents it using charts and tables. This workflow enables the system to transform raw visual feed data into structured insights that can be analysed and compared across different experimental conditions.

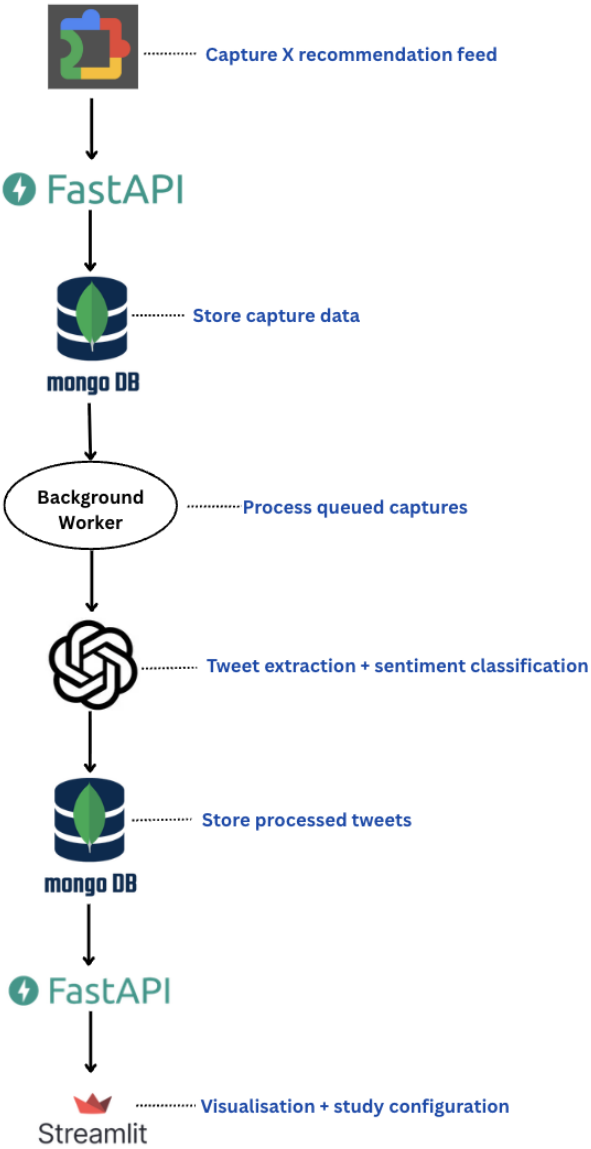

Figure 4.13: End-to-end workflow of the EchoChamber system.

Chapter 5

System Evaluation

This chapter evaluates the EchoChamber system in relation to the aims and objectives defined in Chapter 1. The evaluation considers system functionality, performance, data processing accuracy, and the platform's effectiveness in supporting longitudinal analysis of recommendation feeds.

To assess the analytical capabilities of the system, data collected from the controlled experimental accounts was examined following the completion of each study phase. While the accounts were initially designed to remain identical in all key aspects except for gender-based identity signals, controlled interactions were introduced in later phases, such as posting, liking, and following accounts, to simulate realistic user behaviour. These interactions were applied systematically across both accounts, allowing any observed differences in the resulting feeds to be examined in relation to the defined experimental conditions, rather than uncontrolled user behaviour. This provides a structured basis for evaluating whether the system successfully supports comparative analysis of recommendation outputs. As this study is based on a limited controlled case involving two accounts, the evaluation is intended as a proof of concept of the platform and methodology rather than as ground truth regarding bias in the X recommendation algorithm.

5.1 Experimental Results and Observations

5.1.1 Overview of Collected Data

Data was collected across multiple study phases using two controlled accounts designed to differ only in gender-based identity signals. The collection process involved automated capture of the X "For You" recommendation feed using the Chrome extension described in Chapter 4.

In total, 50 capture sessions were recorded over a period of 21 weeks, resulting

in approximately 2300 screenshots. From these captures, 1577 individual tweets were successfully extracted and stored in the database. The dataset was distributed across both accounts, with 752 tweets collected from the male account and 747 from the female account.

Each stored tweet contained structured attributes including the tweet text, username, engagement metrics (likes and retweets), and derived analytical fields such as political leaning, emotional tone, and moral stance. These fields were generated during the processing stage using the backend pipeline.

Prior to analysis, the data was normalised and deduplicated to remove repeated or near-identical tweets, ensuring that the dataset more accurately reflected the diversity of content presented within the recommendation feed. As a result, the final dataset was considered suitable for comparative and longitudinal analysis across study phases.

While the dataset provides a structured basis for analysis, it remains limited by the capture methodology, which relies on periodic screenshots and may not fully represent all content presented within the feed.

5.1.2 Political Leaning Analysis

The following figures present the distribution of political leaning classifications for each phase across both the male and female accounts. Each phase was designed to introduce a controlled change to the accounts while maintaining all other conditions as consistent as possible. The results highlight observable differences in the composition of recommended content between the two accounts under these controlled experimental conditions.

Later phases introduced additional identity and behavioural signals, including explicit gender attributes, gendered usernames, and controlled posting and engagement activity. These modifications provide a structured basis for exploring how incremental changes in account characteristics may relate to variations in the distribution of recommended content over time, without establishing causal relationships.

Phase 1 – Gender Baseline

Phase 1 involved the initial creation of both accounts. During account setup, X prompted the user to select areas of interest. The same topics were selected for both accounts, including News, Movies & TV, Technology, Religion, Science, Education, Business & Finance, Career, Health & Fitness, and Memes. These topics were intentionally chosen to provide a broad and varied set of initial interests, allowing observation of the type of content the recommendation system would present at the outset under shared baseline conditions. The usernames assigned

were ‘boyuser112‘ and ‘girluser112‘, ensuring that both accounts were identical in all aspects except for gender-associated identity signals. This phase therefore establishes a controlled baseline for comparison.

As shown in Figure 5.1, the majority of content presented in phase 1 to the male account (‘boyuser112‘) was classified as apolitical, accounting for approximately 72% of all captured posts. In comparison, the female account (‘girluser112‘) exhibited a lower proportion of apolitical content at 56.5%, indicating a relatively broader distribution of politically classified material.

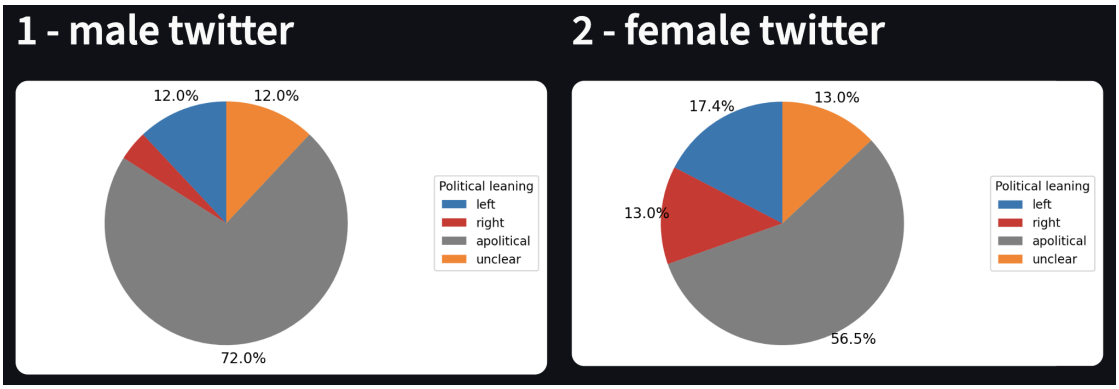

Figure 5.1: Political leaning distribution for male and female accounts in Phase 1.

In terms of explicitly political categories, the female account showed higher representation across both left and right-leaning content. Right-leaning content accounted for approximately 13% of the feed for ‘girluser112‘, compared to 4% for ‘boyuser112‘. Similarly, left-leaning content was more prominent in the female account at 17.4%, compared to approximately 12% for the male account.

The proportion of content classified as unclear was similar for both accounts, with approximately 12% for ‘boyuser112‘ and 13% for ‘girluser112‘, indicating a comparable level of ambiguity in classification across both feeds. From observation, tweets were typically classified as unclear when they referenced political topics without expressing a clear ideological stance. This suggests that the classification process distinguishes between politically relevant content and explicitly partisan or ideologically aligned content, with neutral or descriptive posts more likely to be assigned to the unclear category.

Overall, the results from Phase 1 indicate that differences in the distribution of politically classified content were observable between the two accounts under baseline conditions. The female account displayed a more varied spread across political categories, while the male account feed remained more strongly dominated by apolitical material. At this stage, these observations serve as an initial point

of comparison for subsequent phases, demonstrating how the system captures and reflects variation in recommendation outputs under controlled starting conditions.

Phase 2 – Gender Attribute Added

Phase 2 introduced the addition of explicit gender information to each account while maintaining all other variables from Phase 1. The account 'boyuser112' was assigned the gender male, while 'girluser112' was assigned female. This allows for the evaluation of whether platform-level gender identification influences recommendation outputs.

As shown in Figure 5.2, the majority of content presented to the male account remained classified as apolitical, increasing to approximately 80.2% of all captured posts. In comparison, the female account again exhibited a lower proportion of apolitical content at 63.5%. Relative to Phase 1, this represents an increase in apolitical content for both accounts, with a more pronounced rise observed for the male account.

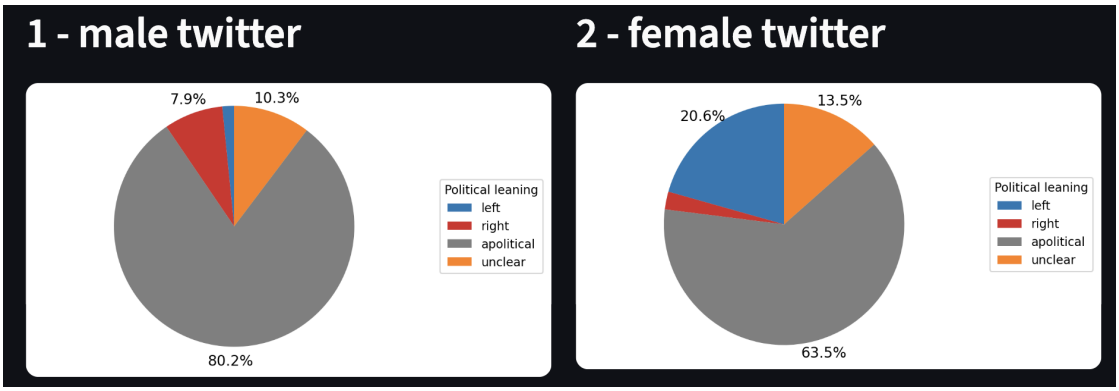

Figure 5.2: Political leaning distribution for male and female accounts in Phase 2.

In terms of explicitly political categories, the distribution differed between accounts. Right-leaning content accounted for approximately 7.9% of the feed for the male account, compared to 2.4% for the female account, indicating a notable reduction in right-leaning content within the female account relative to Phase 1. Conversely, left-leaning content was more prominent in the female account at 20.6%, compared to approximately 1.6% for the male account. This reflects an increase in left-leaning content for the female account and a reduction for the male account following the introduction of gender attributes.

The proportion of content classified as unclear remained broadly similar across both accounts, with approximately 10.3% for the male account and 13.5% for the female account, indicating a comparable level of classification ambiguity.

Overall, the results from Phase 2 show a divergence in the distribution of political content between the accounts following the introduction of gender attributes. The female account exhibited a greater proportion of left-leaning content, while showing a reduction in right-leaning material. The male account retained a higher proportion of right-leaning and apolitical content, while reducing left-leaning material. These observations indicate that differences in content composition were observable following the introduction of gender-related account attributes. While causality cannot be established within the scope of this study, the results demonstrate how the system captures variation in recommendation outputs under controlled changes in account characteristics, providing a basis for exploring how identity-related signals may relate to differences in recommended content.

Phase 3 – Gendered Username Assignment

Phase 3 introduced the assignment of explicitly gendered usernames. The male account was assigned the username 'briansmith2211', while the female account was assigned 'rachelsmith221'. All other variables, including selected interests and prior behaviour, were maintained from Phase 2. This phase allows for the evaluation of whether more realistic identity signals, in the form of common gendered names, influence recommendation outputs.

It should be noted that this period of data capture coincided with a rise in geopolitical tensions between the United States, Israel, and Iran. As a result, the overall increase in politically related content across both accounts may reflect broader external influences on the platform's recommendation system rather than solely the effect of username changes. However, the differing ideological distribution of that content between the two accounts may suggest that username assignment influenced how politically oriented material was presented.

As shown in Figure 5.3, the female account exhibited a substantial shift in the distribution of political content compared to previous phases.

The proportion of apolitical content decreased significantly for the female account to 9.2%, representing a reduction of 54.3% from Phase 2. In contrast, the male account remained relatively stable at 83.9%, a decrease of 3.7%. This indicates a much greater presence of politically classified material within the female account during this phase.

In terms of explicitly political categories, the female account showed a pronounced increase in left-leaning content, rising to 48.8%, an increase of 28.2% from Phase 2. The male account also showed an increase in left-leaning content, though to a much smaller extent, rising to 6.5%, an increase of 4.9%.

Right-leaning content in the male account decreased to 0%, down from 7.9% in Phase 2, while the female account increased to 5.1%, up from 2.4%. This suggests a redistribution of political content across categories, particularly within the female

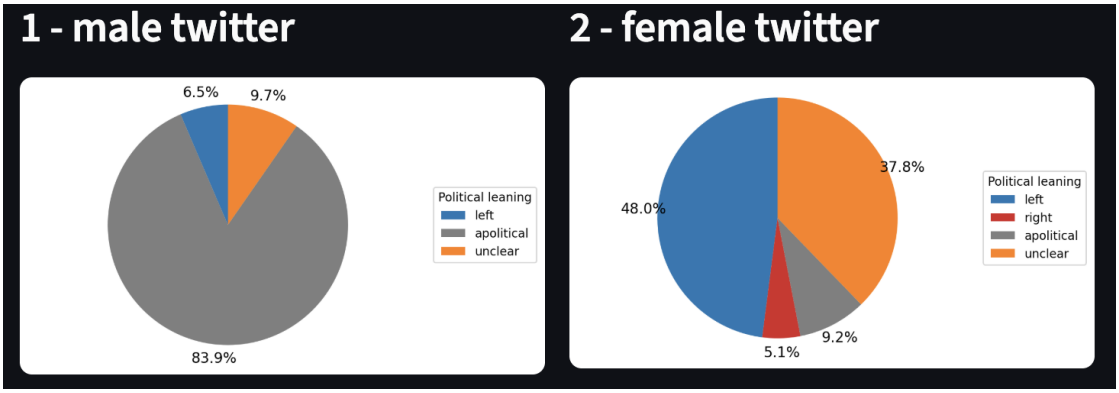

Figure 5.3: Political leaning distribution for male and female accounts in Phase 3.

account.

The proportion of content classified as unclear increased substantially for the female account to 37.8%, representing a rise of 24.3%. This suggests a greater presence of politically relevant commentary that did not align clearly with explicitly left or right-leaning categories. In contrast, the male account remained relatively stable in this category at 9.7%, compared to 10.3% in Phase 2.

Overall, the results from Phase 3 indicate a divergence between the two accounts, with the female account showing a broader and more politically varied distribution of content. While the overall rise in political content may reflect broader geopolitical events during the data collection period, the differing ideological patterns observed between the two accounts suggest that variation in content distribution was present following the introduction of gendered usernames. These observations provide a basis for comparing how recommendation outputs may vary under controlled identity-related changes.

Phase 4 – Posting Activity

Phase 4 introduced the first interaction-driven phase of the experiment. During this phase, both accounts posted the same tweet: "I love politics". This tweet was intentionally chosen to express a general interest in political content without signalling a specific ideological position. The purpose of this phase was to observe whether introducing identical posting behaviour would produce any measurable shift in recommendation outputs while minimising explicit political bias in the posted content.

As shown in Figure 5.4, the female account again exhibited the greater shift in the distribution of political content relative to the male account. The proportion of apolitical content in the female account increased to approximately 22.4%, rep-

resenting a rise of 13.2% from Phase 3. In contrast, the male account remained relatively stable at approximately 85.5%, an increase of 1.6%. This indicates that, although the female account remained substantially more politically characterised than the male account, some movement back towards apolitical content occurred during this phase.

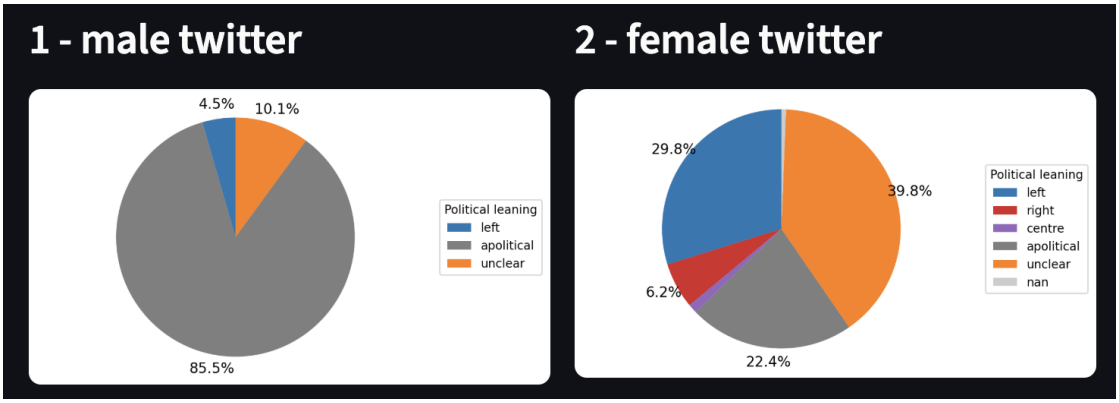

Figure 5.4: Political leaning distribution for male and female accounts in Phase 4.

In terms of explicitly political categories, the female account showed a pronounced decrease in left-leaning content, falling to approximately 29.8%, a reduction of 18.2 percentage points from Phase 3. The male account also showed a decrease in left-leaning content, falling to approximately 4.5%, indicating a similar reduction pattern of left-leaning material within its feed.

Right-leaning content remained at 0% for the male account, while the female account showed a modest increase to approximately 6.2%, representing a rise of 1.1%. This suggests a limited redistribution of political content across categories, particularly within the female account.

The proportion of content classified as unclear increased slightly for the female account to approximately 39.8%, representing a rise of 2%. In contrast, the male account remained relatively stable in this category at approximately 10.1%, indicating limited change in classification ambiguity.

Overall, the results from Phase 4 indicate that the introduction of identical posting behaviour did not correspond to a substantial change in the distribution of content within the male account feed, which remained heavily dominated by apolitical material. The female account, however, continued to display a broader and more politically varied distribution of content, although with some reduction in explicitly left-leaning material compared to Phase 3. These observations provide a further point of comparison for examining how controlled behavioural signals may relate to variation in recommendation outputs.

Phase 5 – Liked Tweet

Phase 5 introduced engagement behaviour through the action of liking a tweet. In this phase, both accounts liked the first tweet that appeared when viewing the profile of Elon Musk. This approach was chosen to introduce a consistent engagement signal across both accounts while minimising additional browsing behaviour that could introduce uncontrolled variables.

Elon Musk was selected due to the prominence of his account within the platform, as it was frequently recommended in the "Who to follow" section and his tweets regularly appeared within the recommendation feed upon account creation. This ensured that interaction with his content reflected a realistic and commonly encountered user scenario.

The tweet liked by both accounts is shown in Figure 5.5. ChatGPT was consulted to eliminate human bias and see did this tweet conform to left or right leaning ideologies. The response was, "Right-leaning. It criticises "social justice" which is typically associated with left-leaning ideology, framing it negatively." As such, engaging with this tweet may introduce a signal that could be interpreted by the recommendation system as aligning with right-leaning perspectives.

As shown in Figure 5.6, the introduction of engagement behaviour through liking a tweet resulted in a noticeable shift in the distribution of political content across both accounts when compared to Phase 4.

For the male account, the proportion of apolitical content decreased from approximately 85.5% in Phase 4 to 73.2% in Phase 5, representing a reduction of 12.3%. This indicates a modest increase in politically classified material within the feed. In contrast, the female account exhibited a more substantial increase in apolitical content, rising from 22.4% in Phase 4 to 45.2% in Phase 5, an increase of 22.8 percentage points.

In terms of explicitly political categories, the male account showed an increase in right-leaning content, rising to approximately 8.2%, while left-leaning content remained minimal. The female account, however, demonstrated a more balanced distribution, with right-leaning content increasing to approximately 19.2% from the 6.2% seen in Phase 4 , with left-leaning content accounting for approximately 12.5%, a decrease of 17.3 percentage points. This represents a redistribution of political content across both ideological categories.

The proportion of content classified as unclear decreased for the female account from approximately 39.8% in Phase 4 to 21.2% in Phase 5, suggesting that more content was being classified into clearly defined political categories. For the male account, the unclear category remained relatively stable at approximately 13.4%.

Overall, the results from Phase 5 indicate that following the introduction of engagement behaviour through liking the selected tweet, a noticeable shift in recommendation outputs was observed, particularly in the female account. Most

Figure 5.5: Tweet liked by both accounts during Phase 5.

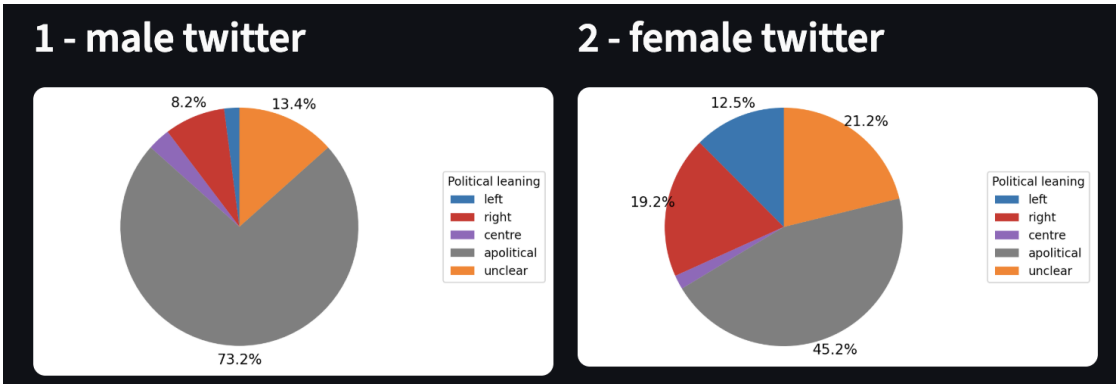

Figure 5.6: Political leaning distribution for male and female accounts in Phase 5.

notably, right-leaning content increased substantially following this interaction, while the proportion of unclear content declined. These observations suggest that engagement with politically suggestive content may correspond to changes in recommendation outputs when compared to earlier phases involving neutral political posting, although the scale and consistency of these changes differed between the two accounts.

Phase 6 – Follow Political Accounts

Phase 6 introduced the following of politically aligned accounts in order to observe how explicit exposure to ideological sources influences recommendation outputs. A selection of three left-leaning and three right-leaning public figures and media accounts was chosen based on their clearly identifiable political alignment. The accounts followed are listed in Table 5.1. Both the male and female accounts followed all of these accounts to ensure consistency in interaction behaviour.

Left-leaning	**Right-leaning**
AOC	Charlie Kirk
Bernie Sanders	Ted Cruz
The Nation	Daily Wire

Table 5.1: Accounts followed in Phase 6

As shown in Figure 5.7, the male account exhibited a substantial shift in the distribution of political content compared to Phase 5. The proportion of apolitical content decreased to 55.7%, representing a reduction of 17.5%. In contrast, the female account remained relatively stable at 44.4%, showing only a marginal decrease of 0.8%. This indicates a greater increase in politically classified material within the male account during this phase.

In terms of explicitly political categories, the male account showed a pronounced increase in right-leaning content, rising to approximately 20%, an increase of 11.8% from Phase 5. Left-leaning content also increased to 7.8%, representing a notable shift from previously low levels.

In comparison, the female account exhibited relatively minor changes. Left-leaning content increased slightly by approximately 0.3%, while right-leaning content rose by 2.2 percentage points. Overall, the distribution for the female account remained more stable, suggesting that the introduction of politically aligned follow behaviour had a more pronounced impact on the male account.

Overall, the results from Phase 6 indicate that following politically aligned accounts was accompanied by an increase in politically classified content, particularly within the male account, where a stronger shift towards right-leaning material was observed. This is notable given that an equal number of left and right-leaning

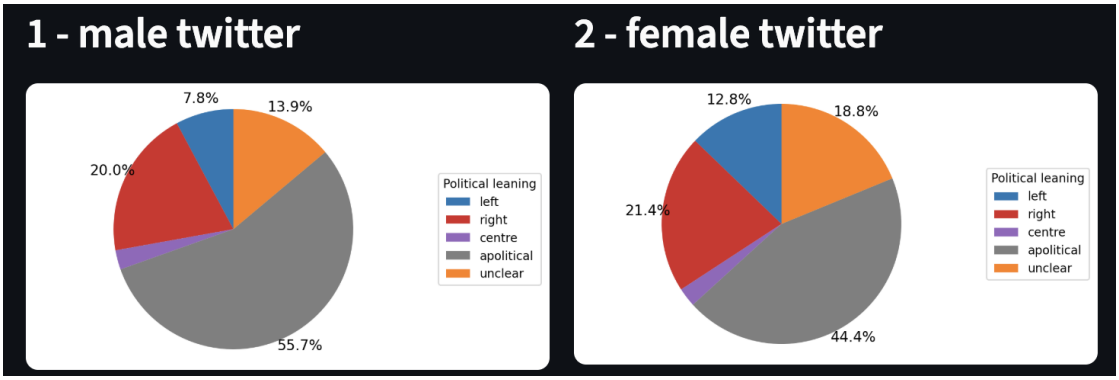

Figure 5.7: Political leaning distribution for male and female accounts in Phase 6.

accounts were followed, where a more balanced distribution of political content might have been expected.

These observations suggest that the distribution of recommended content does not correspond directly to the proportion of followed accounts alone, and that additional factors may be involved in how political content appears within the feed. Within the context of this study, the system captures how such variations emerge under controlled conditions, providing a basis for further exploration rather than establishing causal relationships.

Phase 7 – Strong Engagement Signals

Phase 7 represents the final phase of the experiment and introduced stronger engagement signals by fully reinforcing ideological preferences within each account. In this phase, the accounts were deliberately separated in behaviour, with the male account following only left-leaning accounts and the female account following only right-leaning accounts. This was done to maximise divergence in interaction signals and observe how strongly the recommendation system responds to explicitly one-sided ideological engagement. The full set of accounts followed is shown in Table 5.2.

Left-leaning - Male account	**Right-leaning - Female account**
AOC	Charlie Kirk
Bernie Sanders	Ted Cruz
The Nation	Daily Wire
Robert Reich	Ben Shapiro
Elizabeth Warren	Ron DeSantis

Table 5.2: Accounts followed in Phase 7

As shown in Figure 5.8, both accounts exhibited an increase in apolitical content compared to Phase 6. The male account increased to 68.1%, representing a rise of 12.4%, while the female account increased to 62.7%, representing a rise of 18.3%. This indicates a reduction in the proportion of politically classified content within both accounts during this phase.

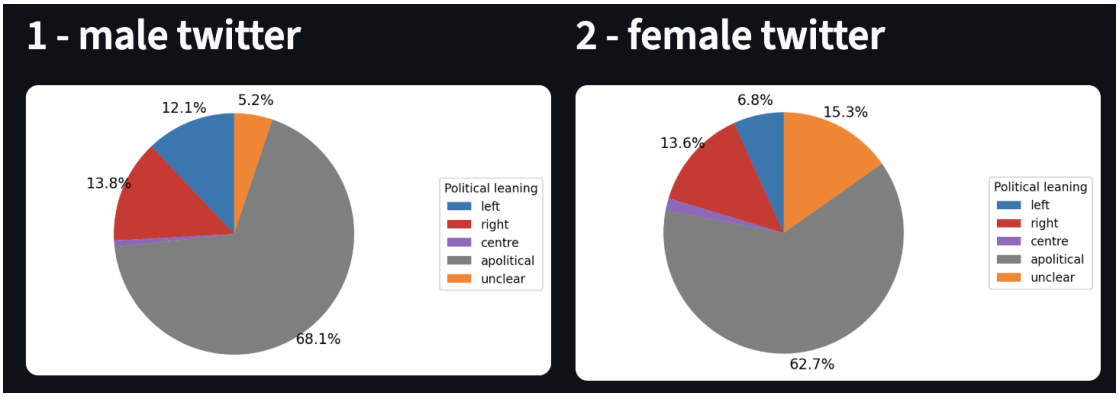

Figure 5.8: Political leaning distribution for male and female accounts in Phase 7.

In terms of explicitly political categories, the male account showed a decrease in right-leaning content, falling to approximately 13.8%, a decrease of 6.2 percentage points from Phase 6. Left-leaning content increased to 12.1%, suggesting a shift towards a more balanced political distribution within the feed.

The female account also exhibited a reduction in right-leaning content, decreasing to approximately 13.6%, while left-leaning content fell to 6.8%. This represents a reversal of the stronger right-leaning presence observed in earlier phases. Additionally, the proportion of content classified as unclear decreased for both accounts, indicating a reduction in politically ambiguous content.

Overall, the results from Phase 7 indicate that stronger and more targeted engagement signals did not correspond to an increase in exposure to politically classified content. Instead, both accounts exhibited a shift towards more apolitical material, suggesting that content distribution during this phase was not solely aligned with explicitly ideological engagement signals.

When focusing specifically on politically classified content, a divergence between expected and observed patterns becomes evident. The male account, which followed only left-leaning accounts, did not exhibit a corresponding dominance of left-leaning content. Instead, its political content remained relatively balanced, with a slight bias towards right-leaning material. This contrasts with the expected outcome, where left-leaning content might have been more prominent.

In contrast, the female account, which followed only right-leaning accounts,

displayed a distribution more closely aligned with its engagement signals, with right-leaning content forming the majority of its political output.

These observations indicate that differences in how recommendation outputs correspond to engagement signals were present between the two accounts. Within the scope of this study, the results highlight that such variations can be captured and analysed under controlled conditions, while also suggesting that additional factors beyond explicit interaction signals may relate to how content is presented, without establishing causal or generalisable conclusions.

5.1.3 Supporting Content Analysis

To complement the overall distribution of political categories, further analysis was conducted using word frequency, topic distribution, and topic-by-leaning bar charts across each phase.

In the early phases (Phase 1-Phase 5), the bar chart visualisations show a clear distinction in the type of content recommended to each account. The male account's top words and topics were consistently dominated by general-interest categories such as sports, entertainment, and casual observations, with very limited representation of explicitly political topics. This is illustrated in Figure 5.9, where the male account's (left side) content is centred around non-political themes.

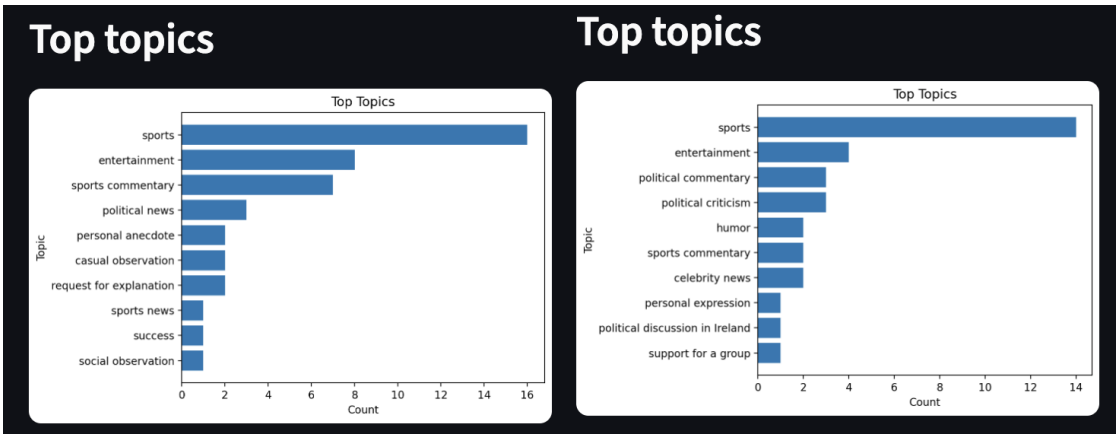

Figure 5.9: Topic distribution for male (left) and female (right) accounts in Phase 2.

In contrast, the female account (right side) displayed a broader and more varied topic distribution from an earlier stage. As shown in Figure 5.9, political topics such as political commentary, international relations, and social issues appear more frequently within the female account's top topics. This suggests that, even when

both accounts were configured similarly, differences in exposure to politically relevant content were observable at an early stage.

The word frequency charts reinforce this distinction. The male account's most frequent terms were largely neutral or entertainment-focused, whereas the female account included a higher presence of politically and socially relevant language. This suggests that differences between the accounts were not only visible in overall category proportions but also in the underlying thematic content.

A significant shift occurs in Phase 6, where both accounts followed an equal number of left-leaning and right-leaning political accounts. The charts for this phase show a marked increase in political topics within the male account, including the emergence of political commentary and geopolitical themes that were largely absent in earlier phases. As shown in Figure 5.10, political topics become substantially more prominent within the male account for the first time. This suggests that more pronounced engagement signals were present in later phases where an increase in political content was observed within the male account. In contrast, the female account shows a more incremental change, as political topics were already present in earlier phases.

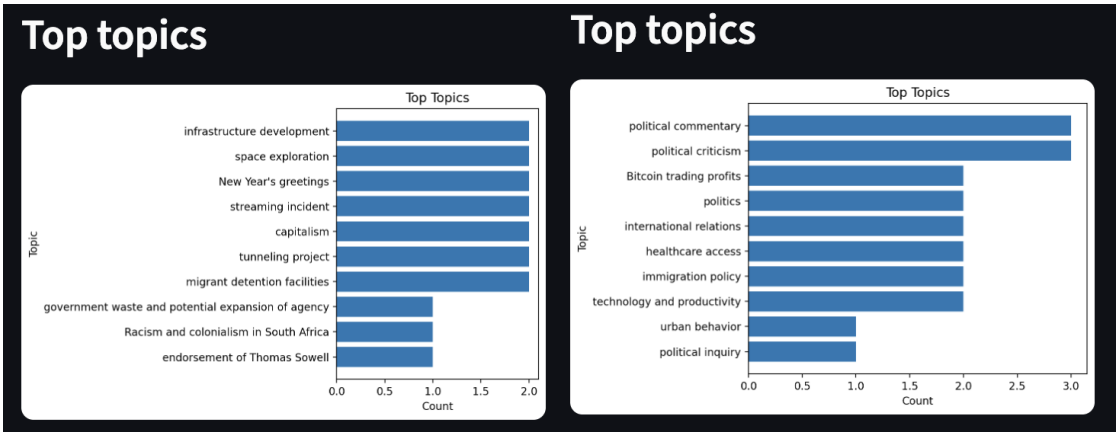

Figure 5.10: Topic distribution for male (left) and female (right) accounts in Phase 6.

In the final phase (Phase 7), although both accounts exhibit an increase in apolitical content overall, the charts reveal that structural differences persist. The female account continues to display a higher concentration of politically oriented topics aligned with its engagement signals, while the male account presents a more mixed and less clearly aligned distribution of topics. This is demonstrated in Figure 5.11, where the female account shows stronger alignment with its right-leaning inputs, while the male account does not exhibit a comparable level of

alignment with its left-leaning signals.

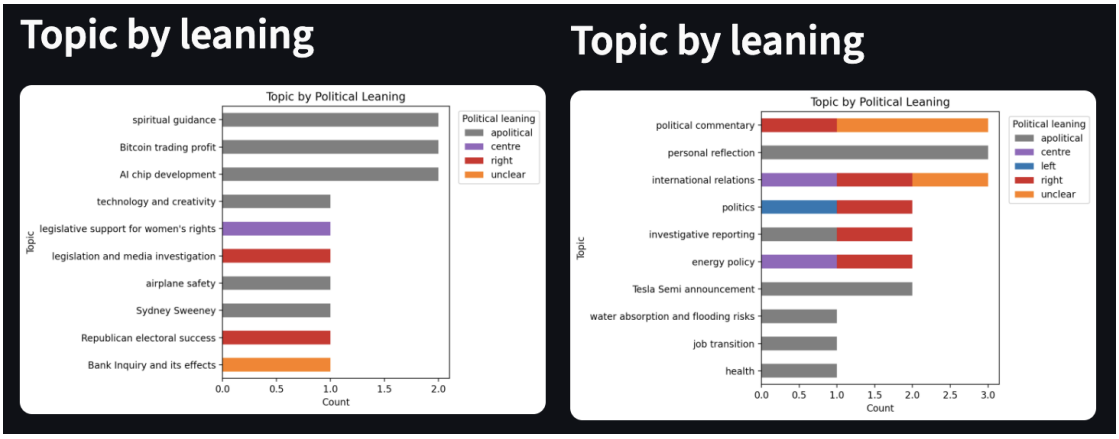

Figure 5.11: Topic by leaning distribution for male (left) and female (right) accounts in Phase 7.

Overall, the bar chart analysis demonstrates that observable differences between the accounts extend beyond the proportion of political content to the nature and timing of topic exposure. The female account displayed politically relevant content earlier and more consistently across phases, whereas the male account remained largely apolitical in the early stages and only exhibited a noticeable increase in political content in later phases following stronger engagement signals. These observations suggest that variation in recommendation outputs may occur not only in magnitude but also in how and when different types of content appear within the feed. Within the scope of this study, the system captures these patterns under controlled conditions, providing a basis for exploring differences in content exposure without establishing causal or generalisable conclusions.

5.2 System Performance Evaluation

This section evaluates the technical performance of the EchoChamber system. The evaluation focuses on the reliability of the data capture process, the efficiency of backend processing, and the overall scalability of the system. These aspects are important in determining whether the platform can consistently support longitudinal data collection and analysis.

Capture Performance

The capture mechanism demonstrated reliable performance, with the Chrome extension consistently scrolling and capturing content from the recommendation feed without interruption during capture sessions. Simultaneously, the use of non-uniform scrolling behaviour, including randomised speed and pauses, allowed the system to mimic human interaction patterns. This reduced the likelihood of the system being detected as automated behaviour and allowed for more naturalistic data collection. However, as the capture process relies on periodic screenshots rather than direct access to platform data, it is possible that some content was not captured between intervals. This may result in partial representation of the full recommendation feed.

The capture process remained stable throughout testing, with no major interruptions or failures observed during operation. This indicates that the extension is suitable for sustained data collection over extended periods.

Backend Performance

The FastAPI backend demonstrated stable performance in handling incoming requests from the Chrome extension, with successful receipt and processing of uploaded capture data. The system also effectively processed captured images, extracting relevant content and converting it into structured data suitable for storage and analysis. However, the use of batch processing and asynchronous workflows introduced some latency between data capture and availability for analysis. While acceptable for this study, this may limit real-time applications.

Scalability

The system performed effectively within the scope of this study, handling the data volume generated by two controlled accounts without observable performance degradation. However, the system has not been tested under high-load conditions. As such, its scalability for larger deployments remains uncertain.

In addition, the use of external APIs for data processing introduces cost-related constraints. As this project was conducted within the context of a university study without dedicated funding, the volume of data processed was necessarily limited. This restricts the ability to scale the system to larger datasets or longer collection periods without incurring increased operational costs.

Future improvements could include optimisations to data processing and storage mechanisms, as well as the introduction of distributed processing to support larger-scale data collection.

5.2.1 System Testing

This section evaluates the testing carried out on the EchoChamber system to ensure correct functionality, reliability, and integration between system components. Testing was conducted across key areas including individual system features, interaction between components, and handling of potential errors.

Functional Testing

Functional testing was conducted to verify that the core features of the system operated as expected. This included user authentication, data capture through the Chrome extension, backend processing, and data visualisation within the frontend.

The login and authentication system successfully validated user credentials and restricted access to authorised users. The Chrome extension was able to initiate and stop capture sessions, correctly sending captured data to the backend. The backend API processed incoming data and stored it in MongoDB without errors. Additionally, the Streamlit frontend successfully retrieved and displayed processed data through charts and visualisations.

Overall, all major system functions operated correctly during testing, indicating that the platform meets its functional requirements.

Integration Testing

Integration testing was carried out to ensure that all system components interacted correctly as a complete pipeline. The system architecture involves multiple stages, including data capture, data transmission, processing, storage, and visualisation. During testing, the Chrome extension successfully captured feed data and transmitted it to the FastAPI backend. The backend processed the data and stored it within MongoDB, after which it was retrieved and displayed through the Streamlit frontend. The integration with the OpenAI API for content extraction and classification also functioned as expected.

The successful interaction between these components demonstrates that the system operates as an end-to-end data collection and analysis platform.

Error Handling

Testing also considered how the system responded to errors and external service failures. Particular challenges were encountered during the implementation of email-based functionality, including password reset and account verification. Initial attempts to integrate SMTP-based email services resulted in repeated failures when sending emails. These issues required investigation through backend error logs provided by the Railway deployment environment. The logs indicated that

requests to the email service were being rejected despite correct implementation. To resolve this, an alternative email service (Brevo) was integrated. However, similar issues persisted until it was identified that the email service account had not been fully validated. This required external resolution through communication with the service provider, after which email functionality operated as expected.

The system was able to handle these failures without crashing, returning appropriate error responses when email delivery was unsuccessful. This demonstrates that while the system is dependent on external services, it remains stable under failure conditions and can recover once configuration issues are resolved.

5.2.2 Limitations of System and Study

Data Collection Limitations

The system relies on screenshot-based capture rather than direct access to platform APIs. As a result, data collection is dependent on what is visible on screen at the time of capture, meaning that some content may not be recorded between capture intervals. This may lead to an incomplete representation of the full recommendation feed.

Model Limitations

The classification of political leaning and sentiment is dependent on responses generated by a large language model. While this enables the transformation of unstructured data into structured outputs, the classification process is not guaranteed to be fully accurate or consistent. No ground truth dataset was available to validate the correctness of classifications, which introduces uncertainty into the analysis.

Model Limitations

The classification of political leaning and sentiment is dependent on responses generated by a large language model. While this enables the transformation of unstructured data into structured outputs, the classification process is not guaranteed to be fully accurate or consistent. No ground truth dataset was available to formally validate the correctness of classifications, which introduces a degree of uncertainty into the analysis. However, manual inspection of a representative sample of classified tweets indicated that the model's outputs were generally aligned with widely accepted interpretations of political leaning. In most cases, tweets were categorised in a manner consistent with their apparent ideological stance, suggesting that the model provided a reasonable approximation of left and right-leaning classifications.

Despite this, the absence of formal validation means that some level of misclassification may be present. As a result, the findings should be interpreted as indicative of broader trends rather than precise measurements of political orientation.

Experimental Limitations

The study was conducted using two controlled accounts, which limits the generalisability of the findings. Additionally, the number of phases and the duration of data collection were constrained by time and resource limitations. External factors, such as real-world events, also influenced the content presented within the feeds, making it difficult to fully isolate the impact of individual variables.

Furthermore, the number of tweets captured and stored varied between phases due to inconsistencies encountered during data processing. In the early stages of the project, the API used for content extraction occasionally returned malformed or incomplete outputs, likely due to variability in model-generated responses. As a result, some extracted tweets could not be reliably structured and were subsequently removed from the dataset. Although improvements were made to the processing pipeline, time constraints limited the ability to fully reprocess earlier data or ensure complete consistency across all phases. Consequently, the volume of usable data differs between phases, which may affect the comparability of results and the overall robustness of the analysis.

This issue was most prominent in Phase 1, where inconsistencies in API responses resulted in a greater proportion of unusable data compared to later phases. However, based on manual observation of the recommendation feed during capture sessions, the data that was successfully collected was considered to be broadly representative of the overall content presented. While some individual posts may not have been captured, the dataset is believed to reflect the dominant patterns within the feed.

Scalability and Cost Limitations

The system was developed within the constraints of a university project without dedicated funding. As a result, the volume of data processed was limited by the cost of external APIs. This restricts the ability to scale the system to larger datasets or longer time periods without additional resources.

Platform Dependency

The system is dependent on the structure and behaviour of the X platform interface. Changes to the user interface or underlying recommendation system may

impact the functionality of the capture mechanism, reducing the reliability of data collection over time. Overall, these limitations highlight that while the system is effective as a research prototype, caution must be taken when interpreting results or extending the system to larger-scale studies.

5.3 Overall Evaluation

Overall, the EchoChamber system successfully demonstrates a working framework for capturing, processing, and analysing recommendation feed data from X. The platform was able to support controlled experimentation across multiple phases, enabling comparative analysis between accounts under varying identity and behavioural conditions. The system achieved its primary aim of supporting longitudinal analysis of recommendation behaviour, with results showing that observable differences in content distribution emerged across phases following changes in account characteristics and engagement patterns. In particular, later phases indicated that variation in recommendation outputs corresponded more closely with engagement signals than with static identity attributes alone. From a technical perspective, the system performed reliably, with stable capture, processing, and visualisation components. Integration between the Chrome extension, backend services, database, and analytical tools was successful, demonstrating the feasibility of the overall architecture. However, the findings of the study must be interpreted within the context of its limitations. The scale of the experiment, reliance on external APIs, and influence of external events mean that conclusions cannot be generalised without further investigation.

In summary, this project demonstrates both a functional system and a viable methodological approach for studying algorithmic content personalisation. While further refinement and scaling would be required for broader application, the system provides a strong foundation for future research in this area.

Chapter 6

Conclusion

This dissertation explored how identity-related account signals and interaction behaviours may relate to variation in the political composition of content recommended by X. To address this, the EchoChamber platform was developed to capture, process, and analyse recommendation feed data across multiple controlled accounts over time. By applying a longitudinal, multi-phase experimental design, the study demonstrates how such a system can be used to observe and analyse differences in content exposure under controlled experimental conditions.

6.0.1 Core Findings

At the outset of the study in Phase 1, the male account was predominantly exposed to apolitical content, with only a small proportion of politically classified material, primarily leaning left or categorised as unclear. In contrast, the female account exhibited a more politically diverse feed from the beginning, with a stronger presence of left-leaning content.

Following the introduction of explicit gender signals in Phase 2, a divergence in content distribution was observed between the accounts. The female account showed a reduction in right-leaning content alongside an increase in left-leaning material, while the male account became increasingly dominated by apolitical content and displayed a shift towards a higher proportion of right-leaning posts. These observations represent an early instance of variation in recommendation outputs following the introduction of gender-related account signals.

Further divergence was observed in Phase 3, when gendered usernames were introduced. During this phase, the female account's feed became more strongly left-leaning, while the male account reduced right-leaning content and shifted towards left-leaning material. Subsequent interaction signals in Phase 4, such as engagement with politically oriented posts, were accompanied by additional variation between the accounts. The female account moved towards a more balanced

distribution between left and right-leaning content, whereas the male account again displayed a higher proportion of right-leaning material, despite remaining largely apolitical overall.

A more pronounced shift was observed in Phase 6, where both accounts followed an equal number of left and right-leaning political accounts. While both feeds showed an increase in politically classified content, the male account exhibited a stronger shift towards right-leaning material, despite the balanced nature of the followed accounts. In contrast, the female account remained relatively stable, continuing to display a higher proportion of left-leaning content.

In the final phase, where the accounts followed opposing ideological groups, this asymmetry persisted. The male account continued to display a mixed distribution of political content, without clear alignment to its configured inputs, whereas the female account showed a distribution more closely aligned with its engagement signals. These findings highlight that observable differences in how recommendation outputs correspond to account configurations were present across phases, as captured by the system under controlled experimental conditions.

Topic-level analysis further supports these findings. In Phase 2, the female account displayed a higher proportion of politically oriented topics compared to the male account, whose feed remained more general and entertainment-focused. By Phase 6, political topics became prominent in both feeds, though more strongly concentrated within the female account. Additionally, the topic-by-leaning breakdown in Phase 7 shows that political topics were increasingly aligned with specific ideological categories, particularly within the female account, suggesting a narrowing of content diversity within the observed data.

Overall, these observations indicate that recommendation outputs did not correspond uniformly to identical user interactions across accounts. While both accounts were subject to the same controlled inputs, differences in content distribution were observed across key phases. The male account displayed a recurring pattern of right-leaning content in certain phases, whereas the female account was exposed to politically oriented content earlier and more consistently throughout the study. These patterns suggest that variation in recommendation outputs may relate to account-specific signals and engagement behaviour. Within the scope of this study, the system captures and illustrates these differences under controlled conditions, without drawing causal or generalisable conclusions.

6.0.2 System Evaluation

In addition to the experimental findings, the EchoChamber system itself was successfully designed and implemented to support large-scale, longitudinal analysis of recommendation feeds. The platform enabled automated data capture through a Chrome extension, structured storage within a MongoDB database, and asyn-

chronous processing using a multi-stage FastAPI pipeline. This architecture allowed for efficient handling of high-frequency data collection while maintaining separation between ingestion and analysis stages. The integration of large language models for tweet extraction and sentiment classification further enabled the transformation of unstructured visual data into structured analytical outputs. Overall, the system met its core objectives by providing a reliable and scalable framework for capturing, processing, and analysing recommendation data, demonstrating its suitability as both a research tool and a foundation for future work in auditing algorithmic personalisation systems.

6.0.3 Future Work

Future work can be considered in two areas: improvements to the system and extensions to the experimental study.

From a system perspective, several enhancements could improve the robustness and scalability of the platform. The current processing pipeline relies on large language models for tweet extraction and sentiment classification, which introduces potential inconsistencies. Future work could incorporate a ground truth labelled dataset to validate and benchmark classification accuracy, or integrate hybrid approaches combining rule-based and model-based methods. In addition, database storage limitations encountered during the project could be addressed through more efficient data management strategies, such as compression, archiving, or migration to a higher-capacity database tier. The system could also be extended to support real-time monitoring and visualisation of recommendation feeds, allowing for more dynamic analysis.

From a research perspective, the study could be expanded by increasing the number of controlled accounts to improve the generalisability of findings. The current experiment was limited to two accounts, which restricts the ability to draw broader conclusions about platform-wide behaviour. Future studies could also explore additional identity signals, such as age, location, or behavioural patterns, to examine how different user attributes influence recommendation outputs. Extending the duration of data collection would allow for deeper analysis of long-term algorithmic adaptation, while incorporating real user data (with appropriate ethical considerations) could provide further validation of the observed trends.

Finally, future work could investigate cross-platform comparisons, applying the system to other social media platforms to determine whether similar patterns of algorithmic personalisation and content bias exist beyond X. This would provide a more comprehensive understanding of how recommendation systems shape information exposure across the wider digital ecosystem.

Final Reflection

Overall, this project demonstrates that it is possible to build a structured and scalable system for analysing algorithmic personalisation in social media environments. The findings highlight that variation in recommendation outputs was observable across accounts under controlled conditions, with patterns suggesting that both user interaction and account-level signals may relate to differences in political content exposure. These results contribute to a growing body of work seeking to understand how personalised recommendation systems operate in practice and illustrate the value of system-based approaches for examining such behaviour. In doing so, the project underscores the importance of transparency and further investigation in the context of algorithmic content delivery.

Bibliography

[1] LIONEL NGOUPEYOU Tondji. Web recommender system for job seeking and recruiting. *Partial Fulfillment of a Masters II at AIMS*, 2018.

[2] Sunshine Chong and Andrés Abeliuk. Quantifying the effects of recommendation systems. In *2019 IEEE International Conference on Big Data (Big Data)*, pages 3008–3015. IEEE, 2019.

[3] Twitter Team. Twitter's recommendation algorithm. https://blog.x.com/engineering/en$_u s/topics/open - source/2023/twitter - recommendation - algorithm, 2023. Accessed : 2026 - 03 - 03.$

[4] P Bouchaud, D Chavalarias, and M Panahi. Crowdsourced audit of twitter's recommender systems. sci. rep. 13, 16815 (2023).

[5] Allison JB Chaney, Brandon M Stewart, and Barbara E Engelhardt. How algorithmic confounding in recommendation systems increases homogeneity and decreases utility. In *Proceedings of the 12th ACM conference on recommender systems*, pages 224–232, 2018.

[6] VP Shivaani, G Muthupandi, et al. A review of generative ai in recommendation systems. In *2025 International Conference on Data Science, Agents & Artificial Intelligence (ICDSAAI)*, pages 1–5. IEEE, 2025.

[7] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*, pages 173–182, 2017.

[8] Zhihao Liu, Wang Zhou, Ying Tian, and Yujun Yang. Deep neural network technique for accurate recommendation service. In *2024 21st International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP)*, pages 1–4. IEEE, 2024.

[9] Tien T Nguyen, Pik-Mai Hui, F Maxwell Harper, Loren Terveen, and Joseph A Konstan. Exploring the filter bubble: the effect of using recommender systems

on content diversity. In *Proceedings of the 23rd international conference on World wide web*, pages 677–686, 2014.

[10] Muhammad Haroon, Magdalena Wojcieszak, Anshuman Chhabra, Xin Liu, Prasant Mohapatra, and Zubair Shafiq. Auditing youtube's recommendation system for ideologically congenial, extreme, and problematic recommendations. *Proceedings of the national academy of sciences*, 120(50):e2213020120, 2023.

[11] Nicholas Sukiennik, Chen Gao, and Nian Li. Uncovering the deep filter bubble: narrow exposure in short-video recommendation. In *Proceedings of the ACM Web Conference 2024*, pages 4727–4735, 2024.

[12] X Developer Team. X api overview. https://docs.x.com/x-api/overview, 2026. Accessed: 2026-04-03.

[13] ChromiumDev. Chrome extensions. https://developer.chrome.com/docs/extensions/get-started, 2026. Accessed: 2026-04-03.

[14] FastAPI. Fastapi documentation. `https://fastapi.tiangolo.com/`, 2026. Accessed: 2026-04-03.

[15] MondoDB Team. Mongodb documentation. https://www.mongodb.com/docs/development/, 2026. Accessed: 2026-04-03.

[16] Vijayaragavan Pichiyan, S Muthulingam, Sunanda Nalajala, Akhil Ch, Manmath Nath Das, et al. Web scraping using natural language processing: exploiting unstructured text for data extraction and analysis. *Procedia Computer Science*, 230:193–202, 2023.

[17] Karthick Prasad Gunasekaran. Exploring sentiment analysis techniques in natural language processing: A comprehensive review. *arXiv preprint arXiv:2305.14842*, 2023.

[18] OpenAI. Openai documentation. https://developers.openai.com/api/docs, 2026. Accessed: 2026-04-06.

[19] Aiswarya Raj, Jan Bosch, Helena Holmström Olsson, and Tian J Wang. Modelling data pipelines. In *2020 46th Euromicro conference on software engineering and advanced applications (SEAA)*, pages 13–20. IEEE, 2020.

[20] Teemu Koivisto. Efficient data analysis pipeline. In *Data science for natural sciences seminar*, pages 1–4, 2019.

[21] Stephen R Midway. Principles of effective data visualization. *Patterns*, 1(9), 2020.

[22] Streamlit Team. Streamlit documentation. https://docs.streamlit.io/, 2026. Accessed: 2026-04-06.

[23] Isaac Odun-Ayo, M Ananya, Frank Agono, and Rowland Goddy-Worlu. Cloud computing architecture: A critical analysis. In *2018 18th international conference on computational science and applications (ICCSA)*, pages 1–7. IEEE, 2018.

[24] Hiral B Patel and Nirali Kansara. Cloud computing deployment models: A comparative study. *International Journal of Innovative Research in Computer Science & Technology (IJIRCST)*, 2021.

[25] Railway. Railway documentation. https://docs.railway.com/, 2026. Accessed: 2026-04-06.

[26] Streamlit. Streamlit community cloud documentation. https://docs.streamlit.io/deploy/streamlit-community-cloud, 2026. Accessed: 2026-04-06.

[27] JSON.org. Introducing json. `https://www.json.org/json-en.html`, 2026. Accessed: 2026-04-07.

[28] OpenAPI Initiative. What is openapi? `https://www.openapis.org/what-is-openapi`, 2026. Accessed: 2026-04-07.